



UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

FACULTAD DE INGENIERÍA

PROGRAMA ACADÉMICO DE CIENCIAS DE LA COMPUTACIÓN

CURSO: 1ACC0065 - PROGRAMACIÓN CONCURRENTES

NRC: 8646

TP

INTEGRANTES:

Manchay Paredes, Lucero Salome (U202216120)

Mayhua Hinostroza, José Antonio (U202218044)

Peña Cardenas, Jhamil Brijan (U201714492)

ÍNDICE

1. Resumen del trabajo.....	4
2. Objetivos del trabajo	5
2.1 Objetivo general	5
2.2 Objetivos específicos	5
3. Explicación de los temas investigados	6
3.1 Concurrencia en Go aplicada a Machine Learning	6
3.2 Regresión Lineal concurrente	6
3.3 Cadena de búsqueda utilizada.....	6
3.4 Tabla de papers por integrante.....	7
Paper 1 (Integrante 1 - José Antonio Mayhua Hinostroza).....	7
Paper 2 (Integrante 2 - Lucero Salome Manchay Paredes).....	9
Paper 3 (Integrante 3 - Jhamil Brijan Peña Cardenas)	10
Elección final del paper de referencia:	13
4. Análisis del caso de uso.....	13
4.1 Elección y sustentación	13
4.2 Selección del dataset	14
5. Procedimiento de limpieza del dataset	15
5.1 Pasos de depuración aplicados.....	15
5.2 Correcciones respecto a la PC1.....	16
6. Modelado inicial en Promela.....	17
6.1 Lógica de sincronización	18
6.2 Procesos y canales	18
6.3 Condiciones verificadas.....	19
6.4 Evidencia de ejecución en SPIN	20
7. Implementación en Go.....	24
7.1 Preparación de datos para Regresión Lineal	25
7.2 Implementación secuencial	28
7.3 Implementación concurrente	29
7.4 Patrón Worker Pool	30
7.5 Mecanismos de sincronización utilizados.....	31
8. Metodología de medición	32
8.1 Entorno de ejecución.....	33
8.2 Número de ejecuciones.....	33
8.3 Configuraciones de workers.....	34

8.4 Cálculo de media recortada	35
9. Resultados experimentales	36
9.1 Tiempos de ejecución	36
9.2 Speedup y eficiencia	37
9.3 Comparación secuencial vs. concurrente.....	38
9.4 Métricas del modelo.....	39
10. Análisis de escalabilidad y trade-offs	40
10.1 Comportamiento de la escalabilidad	40
10.2 Rendimientos decrecientes y overhead.....	41
10.3 Trade-offs de la solución concurrente	42
10.4 Estimación de la fracción secuencial (métrica de Karp–Flatt)	42
11. Uso y rendimiento de recursos de cómputo	43
11.1 CPU	43
11.2 Memoria	44
11.3 Punto de equilibrio	45
12. Trabajo en GitHub	47
13. Correcciones respecto a la PC2	48
14. Verificación formal en SPIN	48
14.1 Propiedades verificadas	48
14.2 Resultados del modelo correcto	48
14.3 Modelos mutantes	49
15. Análisis del código con inteligencia artificial	49
15.1 Prompt y modelo utilizados.....	49
15.2 GAPs encontrados.....	50
15.3 Corrección de los GAPs.....	50
16. Conclusiones y recomendaciones	51
16.1 Conclusiones	51
16.2 Recomendaciones	51
16.3 Limitaciones	51
17. Anexos	51
18. Referencias bibliográficas (formato APA).....	52

1. Resumen del trabajo

El presente trabajo desarrolla una solución de Regresión Lineal aplicada al dataset Hospital Inpatient Discharges (SPARCS De-Identified): 2022, con el objetivo de comparar el rendimiento de una implementación secuencial y una implementación concurrente en Go. A partir del dataset limpio obtenido en la PC1, compuesto por 2,103,432 registros válidos, se realizó el preprocesamiento de las variables, obteniendo 49 características numéricas y una partición reproducible de 79.99% para entrenamiento y 20.01% para prueba. El modelo fue entrenado mediante Batch Gradient Descent utilizando 100 épocas y una tasa de aprendizaje de 0.01.

Para la versión concurrente se implementó un patrón Worker Pool basado en goroutines, channels y sync.WaitGroup, donde cada worker calcula gradientes parciales sobre una

partición de los datos y posteriormente estos resultados son agregados antes de actualizar los parámetros del modelo. La lógica de sincronización fue representada mediante Promela y verificada con SPIN, cuya exploración exhaustiva alcanzó 143,799 estados y 241,264 transiciones sin detectar errores (errors: 0).

La evaluación experimental se realizó mediante siete ejecuciones por configuración y el cálculo de una media recortada, comparando la versión secuencial con configuraciones de 1, 2, 4, 8, 12, 16, 24 y 32 workers. La implementación concurrente alcanzó su menor tiempo medio con 16 workers, con 382.95 ms y un speedup de 4.88x frente a los 1.87 s de la versión secuencial. Sin embargo, el análisis de escalabilidad y recursos mostró una zona de saturación entre 12 y 16 workers, identificándose 12 workers como un punto de equilibrio práctico al coincidir con los 12 procesadores lógicos disponibles y ofrecer un rendimiento cercano al máximo con menor overhead.

Finalmente, ambas implementaciones produjeron resultados equivalentes, con una diferencia máxima entre sus parámetros de 0.000000000000 y un R^2 de 0.467884 sobre el conjunto de prueba, evidenciando que la estrategia de concurrencia mejoró el tiempo de ejecución sin modificar el resultado matemático del modelo.

En el TP, el trabajo se complementó con tres frentes. Primero, la verificación formal en SPIN de la ausencia de deadlocks y de la exclusión mutua, incluyendo modelos mutantes con defectos introducidos a propósito que SPIN detectó, lo que demuestra que la verificación es significativa. Segundo, un análisis del código con inteligencia artificial mediante un prompt estructurado, que identificó 19 GAPS; los de severidad media se corrigieron aplicando TDD. Tercero, las conclusiones y recomendaciones del grupo sobre escalabilidad, limitaciones y mejoras posibles.

2. Objetivos del trabajo

2.1 Objetivo general

Implementar y evaluar una solución de Regresión Lineal secuencial y concurrente en Go sobre el dataset SPARCS 2022, aplicando mecanismos de sincronización y un patrón Worker Pool, con el propósito de analizar el impacto de la concurrencia en el tiempo de ejecución, speedup, escalabilidad y uso de recursos de cómputo.

2.2 Objetivos específicos

- Preparar los datos del dataset SPARCS 2022 para el entrenamiento de un modelo de Regresión Lineal, mediante codificación de variables, partición Train/Test y escalamiento.
- Implementar una versión secuencial de Regresión Lineal mediante Batch Gradient Descent que funcione como línea base para la evaluación de rendimiento.
- Implementar una versión concurrente equivalente utilizando goroutines, channels, sync.WaitGroup y el patrón Worker Pool para paralelizar el cálculo de gradientes.

- Modelar la lógica de sincronización de la solución concurrente mediante Promela y verificar sus propiedades mediante SPIN.
- Comparar experimentalmente las implementaciones secuencial y concurrente mediante múltiples ejecuciones, media recortada, speedup y eficiencia.
- Analizar la escalabilidad, uso de CPU, memoria y overhead de las configuraciones concurrentes para determinar un punto de equilibrio práctico.

3. Explicación de los temas investigados

El grupo investigó dos ejes temáticos relacionados con el caso de uso: (1) programación concurrente en Go aplicada a Machine Learning y (2) el modelo de Regresión Lineal y sus variantes de paralelización. Cada integrante documentó un paper académico reciente, conformando un total de tres artículos, siguiendo el formato establecido para el trabajo.

3.1 Concurrencia en Go aplicada a Machine Learning

Go ofrece concurrencia nativa mediante goroutines (hilos ligeros gestionados por el runtime) y canales, junto con primitivas de sincronización como `sync.Mutex` (exclusión mutua) y `sync.WaitGroup` (espera de finalización de tareas). Estos mecanismos permiten implementar de forma concurrente el entrenamiento o la inferencia de modelos de ML sin depender de librerías externas, distribuyendo el dataset entre varios workers y agregando resultados parciales, típicamente mediante patrones Worker Pool o Pipeline.

3.2 Regresión Lineal concurrente

La Regresión Lineal, al basarse en operaciones matriciales y sumatorias independientes por bloque de datos, es un candidato natural para una implementación concurrente mediante particionamiento de datos (data partitioning). Cada goroutine puede calcular gradientes o sumas parciales de forma independiente sobre una porción del conjunto de datos y posteriormente comunicar sus resultados mediante channels para realizar una fase de reducción sincronizada antes de actualizar los parámetros del modelo.

La concurrencia permite que múltiples goroutines trabajen sobre particiones del dataset de forma intercalada, mejorando la escalabilidad y el tiempo de ejecución en comparación con una implementación secuencial pura. En hardware multi-núcleo, esta concurrencia se traduce en paralelismo real, logrando un speedup medible.

3.3 Cadena de búsqueda utilizada

Como punto de partida para la búsqueda bibliográfica se consideraron los conceptos Machine Learning, programación concurrente y Regresión Lineal. A partir de ellos, cada integrante refinó la cadena de búsqueda según el enfoque específico de su artículo, incorporando términos relacionados con aprendizaje distribuido, procesamiento paralelo, GPU, Gradient Descent y mecanismos de sincronización.

Ejemplo de refinamiento por integrante:

"Machine learning" AND "linear regression" AND "concurrent programming"

Integrante 1: José Antonio Mayhua Hinostroza

- **Paper:** Linear Regression With Distributed Learning: A Generalization Error Perspective
- **Cadena de búsqueda:** "machine learning" AND "linear regression" AND "distributed learning"

Integrante 2: Lucero Salome Manchay Paredes:

- **Paper:** Recursive SQL and GPU-support for in-database machine learning
- **Cadena de búsqueda:** "machine learning" AND "linear regression" AND ("parallel" OR "GPU") AND "gradient descent"

Integrante 3: Jhamil Brijan Peña Cardenas,

- **Paper:** Consistent Lock-free Parallel Stochastic Gradient Descent for Fast and Stable Convergence
- **Cadena de búsqueda:** "machine learning" AND "stochastic gradient descent" AND ("parallel" OR "lock-free")

3.4 Tabla de papers por integrante

Cada integrante documentó un paper académico reciente, conformando tres artículos en total, provenientes de fuentes académicas indexadas y relacionados con Regresión Lineal, paralelismo, concurrencia o mecanismos de sincronización.

Paper 1 (Integrante 1 - José Antonio Mayhua Hinostroza)

Campo	Contenido
Título del paper	Linear Regression with Distributed Learning: A Generalization Error Perspective
Autores	Martin Hellkvist, Ayça Özçelikkale y Anders Ahlén
Año de publicación	2021
Motivación	El aprendizaje distribuido permite abordar tareas de aprendizaje a gran escala repartiendo la carga computacional entre múltiples nodos de una red. Sin embargo, los autores señalan que gran parte de los estudios existentes se concentra en el rendimiento sobre los datos de entrenamiento y no analiza suficientemente cómo la distribución del modelo afecta su capacidad para generalizar sobre datos no observados. Por ello, el trabajo estudia la Regresión Lineal

		distribuida desde la perspectiva del error de generalización, buscando identificar posibles problemas derivados del particionamiento del modelo.
Problema que resuelve		El trabajo aborda la falta de conocimiento sobre cómo el particionamiento de los parámetros de un modelo de Regresión Lineal entre diferentes nodos afecta el error de generalización. En particular, busca determinar si una solución distribuida que obtiene un bajo error de entrenamiento conserva necesariamente un buen rendimiento sobre datos nuevos. Los autores muestran que esto no siempre ocurre: el error de generalización puede ser bastante mayor que el de una solución centralizada aun cuando ambas tengan errores de entrenamiento similares.
Propuesta de solución		Los autores estudian una Regresión Lineal en la cual los parámetros desconocidos del modelo se dividen entre K nodos. Para obtener la solución distribuida usan el algoritmo COCOA, que permite que cada nodo resuelva un subproblema local y comunique su contribución a un nodo central. Después analizan de forma matemática el error de generalización producido por distintas formas de particionar los parámetros y establecen cotas probabilísticas para datos gaussianos isotrópicos, gaussianos correlacionados y subgaussianos. La propuesta se complementa con experimentos sobre datos sintéticos y datos reales de MNIST.
Algoritmos usados / pseudocódigo		Se usa Regresión Lineal regularizada mediante mínimos cuadrados y el algoritmo/framework de optimización distribuida COCOA. La matriz de regresores se divide por columnas entre K nodos. En cada iteración, cada nodo calcula una actualización de sus parámetros locales mediante un problema de optimización convexa; después envía su contribución a un nodo central, que calcula una variable agregada y la redistribuye a los nodos para iniciar la siguiente iteración.
Servicios, frameworks, bibliotecas, lenguaje, patrones		El artículo no especifica un lenguaje de programación, biblioteca o framework de software concreto como Python, TensorFlow, PyTorch, Go o MATLAB. Su contribución es principalmente matemática y algorítmica. Usa el algoritmo COCOA como método de optimización distribuida y una arquitectura de múltiples nodos con coordinación y agregación central. Para los experimentos usa datos sintéticos y el dataset MNIST. Por ello, no sería correcto atribuirle herramientas de programación que el artículo no declara.
Opinión crítica del estudiante		Considero que el artículo muestra que distribuir un modelo de Machine Learning puede ser beneficioso para repartir la carga computacional de problemas de gran escala, pero también deja claro

	que la distribución debe diseñarse con cuidado. Un aspecto relevante es que un bajo error de entrenamiento no garantiza un bajo error de generalización: dependiendo de cómo se dividan los parámetros entre los nodos, el rendimiento sobre datos nuevos puede empeorar bastante. Esto es importante para nuestro proyecto porque muestra que, al aplicar concurrencia o paralelismo, no basta con aumentar el número de workers para obtener una mejor solución; además del tiempo de ejecución y el speedup, es necesario analizar cómo se divide el trabajo y verificar que la calidad del modelo no se vea afectada.
Link de acceso al paper	DOI: https://ieeexplore.ieee.org/document/9520293 Acceso abierto al paper: https://www.researchgate.net/publication/354044522_Linear_Regression_With_Distributed_Learning_A_Generalization_Error_Perspective

Paper 2 (Integrante 2 - Lucero Salome Manchay Paredes)

Campo	Contenido
Título del paper	Recursive SQL and GPU-support for in-database machine learning
Autores	Maximilian E. Schüle, Harald Lang, Maximilian Springer, Alfons Kemper, Thomas Neumann y Stephan Günnemann.
Año de publicación	2022
Motivación	Los autores identifican que los pipelines tradicionales de Machine Learning suelen extraer los datos de una base de datos hacia herramientas externas como NumPy, TensorFlow o PyTorch para realizar el entrenamiento. Esta extracción puede resultar costosa cuando se trabaja con grandes volúmenes de información. Por ello, investigan la posibilidad de ejecutar directamente dentro del sistema de base de datos las etapas de preprocesamiento, entrenamiento y validación, aprovechando además procesamiento paralelo en CPU y GPU.
Problema que resuelve	Busca reducir el costo de transferencia y procesamiento requerido para entrenar modelos de Machine Learning sobre grandes conjuntos de datos. Además, estudia cómo paralelizar el entrenamiento mediante múltiples threads, workers y GPUs sin que los mecanismos de sincronización reduzcan excesivamente el rendimiento. El artículo utiliza explícitamente Regresión Lineal como uno de sus modelos de evaluación.

Propuesta de solución	Los autores extienden el sistema de base de datos Umbra para expresar un pipeline completo de Machine Learning mediante SQL. Incorporan diferenciación automática y un operador de Gradient Descent capaz de generar código LLVM y ejecutar el entrenamiento sobre GPU. Para Regresión Lineal, los datos se procesan por mini-batches y se distribuyen entre threads/workers. Se comparan estrategias con sincronización global, actualizaciones sin bloqueo y modelos locales.
Algoritmos usados / pseudocódigo	Se utiliza Regresión Lineal, Mini-batch Gradient Descent, diferenciación automática y técnicas paralelas de reducción/agregación de gradientes. El artículo presenta distintas estrategias de sincronización: workers sincronizados, workers con actualizaciones globales y workers con modelos locales. Cada worker procesa una parte de los datos, calcula su gradiente y posteriormente los resultados son agregados para actualizar los pesos del modelo.
Servicios, frameworks, bibliotecas, lenguaje, patrones	Umbra como DBMS experimental; SQL para representar el pipeline; LLVM para generación de código; CUDA para ejecución en GPU y cuBLAS para operaciones vectoriales/matriciales. Se utilizan CPU threads, GPU threads, múltiples workers, particionamiento de datos, mini-batches, reducción de gradientes y modelos globales/locales.
Opinión crítica del estudiante	Consideramos que el artículo evidencia que aumentar el número de workers no garantiza por sí solo un mejor rendimiento, ya que una sincronización inadecuada puede introducir contención y reducir el speedup alcanzable. Esta observación resultó especialmente relevante para nuestra implementación en Go, en la cual se utilizaron goroutines, channels y sync.WaitGroup para coordinar el procesamiento concurrente. Para reducir la contención, cada worker calcula gradientes parciales de forma independiente y posteriormente los resultados son agregados antes de actualizar los parámetros del modelo. De esta manera, se evita realizar actualizaciones concurrentes directas sobre un gradiente global protegido mediante bloqueos por cada registro procesado.
Link de acceso al paper	https://link.springer.com/article/10.1007/s10619-022-07417-7

Paper 3 (Integrante 3 - Jhamil Brijan Peña Cardenas)

Campo	Contenido
Título del paper	Consistent Lock-free Parallel Stochastic Gradient Descent for Fast and Stable Convergence

Autores	Karl Bäckström, Ivan Walulya, Marina Papatriantafilou y Philippos Tsigas (Chalmers University of Technology, Suecia).
Año de publicación	2021 (35th IEEE International Parallel & Distributed Processing Symposium, IPDPS 2021)
Motivación	El Descenso de Gradiente Estocástico (SGD) es el motor de optimización de numerosos algoritmos de Machine Learning, incluida la Regresión Lineal. Los autores observan que, si bien existen variantes asíncronas de SGD sobre memoria compartida (AsyncSGD y HOGWILD!) que reducen el costo de coordinación frente a la parallelización síncrona, la literatura no explica con claridad en qué situaciones prácticas resultan realmente útiles ni qué papel cumplen los mecanismos de sincronización y de consistencia. Señalan además que muchos trabajos previos miden la convergencia en número de iteraciones (eficiencia estadística) y no en tiempo real de reloj, lo que puede llevar a conclusiones engañosas sobre el beneficio verdadero de la concurrencia.
Problema que resuelve	El artículo aborda el compromiso entre consistencia y sobrecarga de sincronización en SGD concurrente sobre memoria compartida. Cuando varios hilos leen y actualizan simultáneamente el vector de parámetros, aparecen dos fenómenos que degradan el aprendizaje: la obsolescencia (staleness), es decir, aplicar un gradiente calculado sobre una versión ya desactualizada de los parámetros, y la inconsistencia, que surge cuando las lecturas y escrituras no coordinadas dejan el vector en un estado intermedio. Usar bloqueos (mutex) elimina la inconsistencia pero introduce contención y espera; eliminarlos por completo (estilo HOGWILD!) elimina la espera pero rompe la consistencia. El trabajo busca determinar si vale la pena pagar el costo de garantizar consistencia y cómo hacerlo sin bloqueos.
Propuesta de solución	Los autores proponen Leashed-SGD, un marco algorítmico extensible de implementaciones de AsyncSGD que preservan la consistencia empleando sincronización lock-free (libre de bloqueos) en lugar de exclusión mutua. El componente central es la estructura de datos ParameterVector, que encapsula el vector de parámetros junto con metadatos de control (número de secuencia, contador de lectores activos y banderas de obsolescencia y borrado) y permite reciclar memoria de forma segura. Cada hilo calcula su gradiente localmente y publica el resultado mediante una única operación atómica Compare-And-Swap (CAS) sobre un puntero global. Además incorporan un umbral de persistencia (T_p) que limita el número de reintentos de CAS y actúa como mecanismo autorregulador de la contención.

Algoritmos usados / pseudocódigo	Se emplean SGD secuencial, AsyncSGD basado en bloqueos (mutex), HOGWILD! (sin sincronización) y el propuesto Leashed-SGD. El artículo presenta pseudocódigo para los cuatro. En AsyncSGD cada hilo bloquea el mutex, copia los parámetros, libera, calcula el gradiente, y vuelve a bloquear para aplicar la actualización. En HOGWILD! se eliminan las llamadas de bloqueo. En Leashed-SGD (Algoritmo 3) el hilo obtiene un puntero al vector más reciente mediante la función <code>latest_pointer()</code> dentro de un bucle de reintento, calcula el gradiente sin copiar, crea una nueva instancia de <code>ParameterVector</code> con la actualización aplicada e intenta publicarla con CAS en el bucle LAU-SPC (<code>LoadAndUpdate-StorePersistenceConditional</code>); si el CAS falla, otro hilo se adelantó y se reintenta hasta superar T_p. Se demuestra formalmente (Lema 1) que las lecturas y actualizaciones cumplen lock-freedom y atomicidad, y (Lema 2) que el reciclaje de memoria es seguro y acotado.
Servicios, frameworks, bibliotecas, lenguaje, patrones	El marco está implementado en C++, usando OpenMP para el cómputo paralelo en memoria compartida y la biblioteca Eigen para las operaciones numéricas, extendiendo la biblioteca de deep learning MiniDNN. Los patrones y mecanismos empleados son: sincronización lock-free mediante operaciones atómicas read-modify-write (CAS y Fetch-And-Add), exclusión mutua con locks como línea base de comparación, bucles de reintento, conteo atómico de lectores, reciclaje dinámico de memoria y particionamiento del trabajo entre hilos. Las pruebas se ejecutaron sobre un Intel Xeon E5-2695 de 36 núcleos con hyper-threading, entrenando MLP y CNN sobre el dataset MNIST con hasta 68 hilos concurrentes.
Opinión crítica del estudiante	Considero que este artículo aporta la lección más directamente aplicable a nuestro trabajo: la concurrencia no mejora el rendimiento por sí sola, sino que su beneficio depende críticamente del mecanismo de sincronización elegido. Los autores muestran que la alternativa aparentemente más rápida, eliminar toda sincronización, como HOGWILD!, se vuelve inestable al aumentar el paralelismo, al punto de que con 68 hilos ninguna ejecución de las líneas base logró converger, mientras que la variante lock-free consistente sí lo hizo y alcanzó mejoras de entre 20-80% y hasta 4x en tiempo real de convergencia, además de reducir el uso de memoria. Esto evidencia un perjuicio importante de la programación concurrente mal diseñada: la contención y la obsolescencia pueden anular por completo la ganancia teórica del speedup. Un aspecto valioso desde el punto de vista metodológico es su insistencia en medir el tiempo real de reloj y no solo el número de iteraciones, criterio que se aplicó

	al comparar nuestras versiones secuencial y concurrente en Go. Como limitación, la evaluación se centra en redes neuronales (MLP y CNN) y no en Regresión Lineal, aunque los propios autores señalan que SGD es la base de modelos convexos como la regresión lineal y que el marco es aplicable a cualquier problema de optimización. Asimismo, aunque el artículo utiliza C++ y OpenMP, sus conceptos sobre contención, consistencia y coordinación entre hilos fueron considerados en el diseño de nuestra implementación en Go mediante goroutines, channels y sync.WaitGroup. En particular, se evitó que múltiples workers modificaran directamente los pesos globales durante el cálculo de gradientes.
Link de acceso al paper	DOI / IEEE Xplore: https://ieeexplore.ieee.org/document/9460457 Acceso abierto al paper: https://arxiv.org/abs/2102.09032

Elección final del paper de referencia:

Tras revisar los tres artículos, el grupo seleccionó como referencia principal el trabajo de Schüle et al. (2022), Recursive SQL and GPU-support for in-database machine learning, debido a que presenta el mayor grado de correspondencia con el caso de uso propuesto. El artículo emplea explícitamente Regresión Lineal y Gradient Descent, distribuyendo el procesamiento de los datos entre múltiples workers y evaluando diferentes estrategias de sincronización y actualización del modelo.

Esta propuesta resultó especialmente relevante para el proyecto debido a que la solución implementada utiliza Regresión Lineal sobre un dataset de gran tamaño y evalúa su procesamiento mediante mecanismos de concurrencia en Go. Aunque las tecnologías empleadas en el artículo, como CPU threads, CUDA y GPU, difieren del entorno utilizado en el proyecto, los principios de particionamiento de datos, cálculo independiente de gradientes y agregación de resultados fueron trasladados conceptualmente al uso de goroutines y un patrón Worker Pool.

El trabajo de Bäckström et al. (2021) se adopta como referencia complementaria para el análisis de sincronización, ya que demuestra que incrementar el paralelismo no garantiza por sí mismo una mejora del rendimiento y que la contención, la consistencia y el mecanismo de actualización de los parámetros influyen directamente en el tiempo de ejecución.

Finalmente, Hellkvist et al. (2021) aporta una perspectiva complementaria sobre el particionamiento en Regresión Lineal distribuida y advierte que una distribución inadecuada del problema puede afectar la capacidad de generalización del modelo.

4. Análisis del caso de uso

4.1 Elección y sustentación

El caso de uso seleccionado consiste en la estimación del costo total de atención hospitalaria a partir de variables clínicas y administrativas asociadas a cada episodio de hospitalización, con el propósito de apoyar la planificación presupuestaria y el análisis del consumo de recursos en centros de salud, en línea con el ODS 3 (Salud y bienestar). Se eligió este caso por tres razones principales:

- Relevancia social: la gestión eficiente de recursos hospitalarios impacta directamente en la calidad y oportunidad de la atención médica, especialmente en sistemas de salud con recursos limitados.
- Naturaleza tabular de los datos: el análisis combina variables numéricas, como Length of Stay, APR Severity of Illness Code y Total Costs, con variables categóricas como Age Group, Type of Admission, APR MDC Description, APR Medical Surgical Description, Payment Typology 1 y Emergency Department Indicator. Esto permite trabajar con información estructurada sin requerir procesamiento de imágenes médicas ni texto clínico libre.
- Escalabilidad del problema: los sistemas hospitalarios generan volúmenes del orden de millones de registros de atención y altas hospitalarias, lo que proporciona un escenario adecuado para evaluar el impacto del procesamiento concurrente sobre grandes volúmenes de datos.

El modelo elegido, Regresión Lineal, se utiliza para estimar el costo total de atención (Total Costs) a partir de la duración de la hospitalización (Length of Stay) y variables clínicas y administrativas asociadas al episodio hospitalario, como severidad, categoría diagnóstica principal, tipo de admisión, grupo etario y tipo de pago. Esta estimación puede contribuir a la planificación presupuestaria y al análisis del consumo de recursos hospitalarios.

4.2 Selección del dataset

Se evaluó inicialmente el dataset público "Medicare Inpatient Prospective Payment System - Provider Summary" (CMS), que contiene variables agregadas por hospital y DRG (total_discharges, average_covered_charges, average_total_payments). Sin embargo, este dataset solo contiene aproximadamente 163,000 registros (cerca de 3,000 hospitales por ~100 categorías de procedimiento), por lo que no cumple el requisito de más de un millón de registros exigido por el curso.

Se evaluó también el CMS Data Entrepreneurs' Synthetic Public Use File (DE-SynPUF), que sí supera holgadamente el millón de registros, pero corresponde a reclamos sintéticos de 2008-2010, fuera de la ventana de vigencia de 5 años que el grupo decidió aplicar de forma consistente tanto a los papers académicos como al dataset.

Por ello, el grupo seleccionó como dataset definitivo el Hospital Inpatient Discharges (SPARCS De-Identified) del año 2022, publicado por el New York State Department of Health (2024) a través del portal público Health Data NY. El SPARCS (Statewide Planning and Research Cooperative System) es un sistema de reporte obligatorio que recopila el detalle de cada alta hospitalaria en el estado de Nueva York, y su versión desidentificada es de acceso público y gratuito.

El dataset SPARCS 2022 contiene 2,103,433 registros a nivel de alta hospitalaria individual. Para el caso de uso se seleccionaron variables relacionadas con duración, características clínicas y administrativas de la hospitalización: Length of Stay, Age Group, Type of Admission, APR Severity of Illness Code, APR MDC Description, APR Medical Surgical Description, Payment Typology 1, Emergency Department Indicator y Total Costs. Esta última constituye la variable objetivo del modelo de Regresión Lineal (New York State Department of Health, 2024).

Como respaldo académico de la elección, Jain et al. (2024) utilizaron una fuente del sistema SPARCS con aproximadamente 2.3 millones de registros desidentificados de altas hospitalarias para desarrollar modelos de Machine Learning orientados a la predicción de la duración de la estancia hospitalaria, lo que respalda la pertinencia de esta fuente para el caso de uso planteado.

5. Procedimiento de limpieza del dataset

El procedimiento de depuración se implementó en Go sobre el dataset Hospital Inpatient Discharges (SPARCS De-Identified): 2022. Las reglas aplicadas fueron definidas a partir del análisis de las características y calidad de los datos, con el objetivo de conservar únicamente la información necesaria para el modelo de Regresión Lineal desarrollado en la PC2 y evitar la eliminación injustificada de hospitalizaciones válidas.

5.1 Pasos de depuración aplicados

- **Carga y lectura:** se procesó el archivo CSV completo correspondiente a SPARCS 2022, compuesto por 2,103,433 registros de altas hospitalarias.
- **Selección de variables:** se conservaron Age Group, Length of Stay, Type of Admission, APR Severity of Illness Code, APR MDC Description, APR Medical Surgical Description, Payment Typology 1, Emergency Department Indicator y Total Costs. Asimismo, se generaron las variables auxiliares LOS_120_plus y Severity_unknown.
- **Conversión de Total Costs:** los separadores de miles y símbolos monetarios fueron eliminados y el valor se convirtió en un dato numérico. Los registros cuyo costo no podía convertirse correctamente o cuyo valor era menor o igual a cero fueron descartados.
- **Tratamiento de Length of Stay:** la estancia hospitalaria fue convertida a un valor entero. Los registros no convertibles o con una estancia menor o igual a cero fueron considerados inválidos.
- **Tratamiento de "120 +":** debido a que esta categoría representa hospitalizaciones de 120 días o más, se almacenó Length of Stay = 120 y se creó la variable binaria LOS_120_plus = 1, conservando así la información de censura presente en el dato original.
- **Tratamiento de severidad:** los valores válidos de APR Severity of Illness Code se conservaron en el rango 1-4. Los códigos 0, vacíos o inválidos se representaron

mediante APR Severity of Illness Code = 0 y Severity_unknown = 1, evitando eliminar esos registros.

- **Variables categóricas:** se eliminaron espacios innecesarios y los valores vacíos se representaron mediante la categoría Unknown.
- **Valores atípicos:** durante el análisis se identificaron valores extremos en costos y días de estancia. Sin embargo, estos no fueron eliminados automáticamente mediante IQR, debido a que una hospitalización prolongada o de alto costo puede representar un caso clínico válido y no necesariamente un error de calidad.
- **Resultado:** de los 2,103,433 registros procesados, se conservaron 2,103,432 y se descartó únicamente un registro por no cumplir la regla de validez establecida para Total Costs. Asimismo, se identificaron y transformaron 2,103 registros con Length of Stay = "120 +" y 636 registros con severidad desconocida. El resultado fue almacenado en SPARCS_2022_clean_go.csv.

Figura 1. Evidencia de la ejecución del proceso de limpieza desarrollado en la PC1 y reutilizado como entrada para la PC2.

```
PS C:\Users\SeuNg720\Documents\Grupo5-ProgramacionConcurrente> go run main.go

SPARCS 2022 - RESULTADO DE LIMPIEZA PC1
Registros leídos:                2103433
Registros conservados:           2103432
Registros descartados:           1
- Total Costs inválido/no > 0:   1
- Length of Stay inválido/no >0: 0
Casos Length of Stay = 120+:     2103
Casos de severidad desconocida:  636

Dataset limpio generado en:
dataset/SPARCS 2022 clean go.csv
```

El archivo SPARCS_2022_clean_go.csv obtenido mediante este procedimiento constituye la entrada utilizada en la PC2 para las etapas posteriores de preprocesamiento, codificación de variables, partición Train/Test y escalamiento, descritas en la sección 7.1.

5.2 Correcciones respecto a la PC1

Con respecto a la versión entregada en la PC1, las secciones 1 a 5 de este informe incorporan los siguientes ajustes:

- **Resumen y objetivos:** se reescribieron para describir el trabajo efectivamente realizado y sus resultados, en lugar de la planificación en tiempo futuro de la PC1.
- **Mecanismos de sincronización (sección 3.2):** la PC1 indicaba que las sumas parciales se combinarían usando sync.Mutex y sync.WaitGroup. La implementación final no utiliza sync.Mutex: los workers envían sus gradientes parciales por channels y la reducción y actualización de pesos se realizan en un único flujo después de sync.WaitGroup.Wait().

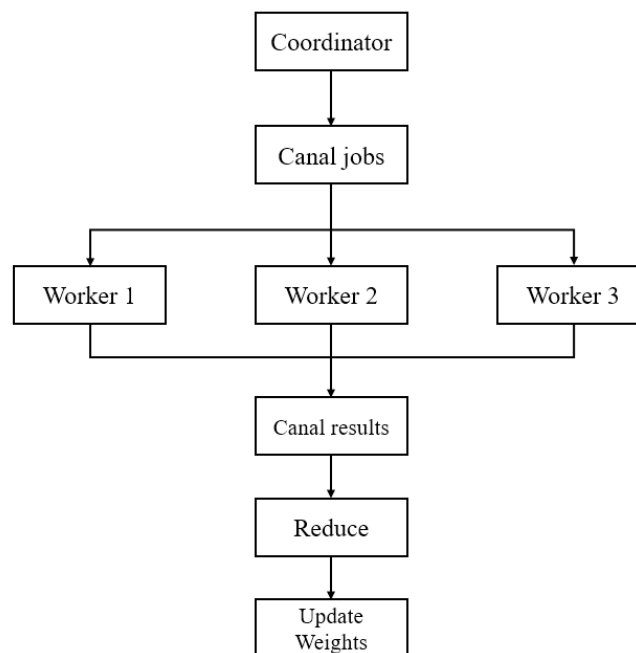
- **Cadena de búsqueda (sección 3.3):** se eliminó el párrafo duplicado que aparecía en la PC1 y se consolidó en una sola explicación.
- **Opiniones críticas de los papers 2 y 3:** se actualizaron para vincularlas con el diseño implementado (gradientes parciales independientes y ningún worker escribiendo directamente sobre los pesos globales), en lugar de la referencia genérica a sync.Mutex.
- **Redacción:** se unificó el uso del tiempo pasado en todo el documento.

6. Modelado inicial en Promela

Como parte del diseño y validación de la solución concurrente, se modeló en Promela la lógica de sincronización utilizada por el patrón Worker Pool. El objetivo del modelo no fue reproducir las operaciones matemáticas completas de la Regresión Lineal ni representar los más de dos millones de registros del dataset, sino abstraer los elementos concurrentes relevantes de la solución: distribución de trabajos, procesamiento por múltiples workers, comunicación de resultados parciales, reducción de dichos resultados y actualización posterior de los parámetros del modelo.

La arquitectura representada corresponde conceptualmente a la implementación concurrente desarrollada en Go:

Figura 2. Arquitectura abstracta del Worker Pool modelada en Promela.



El coordinador distribuye trabajos mediante el canal jobs, los workers procesan las tareas de forma concurrente y comunican sus resultados a través del canal results; posteriormente se realiza la reducción y actualización de los parámetros.

El archivo correspondiente al modelo se almacenó en: *promela/regression_workers.pml*

6.1 Lógica de sincronización

La solución concurrente divide el procesamiento del conjunto de datos en trabajos independientes. Cada worker recibe un trabajo, realiza conceptualmente el cálculo de un gradiente parcial y envía el resultado al coordinador. Los workers no modifican directamente los pesos globales del modelo.

Esta decisión permite separar el cálculo paralelo de la actualización del modelo. La lógica empleada sigue cuatro etapas principales:

- A. El coordinador crea el conjunto de workers.
- B. Los trabajos son enviados mediante el canal jobs.
- C. Cada worker procesa un trabajo y envía su resultado mediante el canal results.
- D. El coordinador recibe todos los resultados parciales, realiza la reducción y solo entonces permite la actualización de los pesos.

En el modelo Promela se utilizan las variables `processed[]` y `received[]` para registrar respectivamente qué trabajos fueron procesados y qué resultados fueron recibidos. Además, `result_count` mantiene el número total de resultados recolectados y `weights_updated` representa el momento en que se realiza la actualización global del modelo.

La actualización de pesos se encuentra deliberadamente fuera de los procesos Worker. De esta forma, la abstracción evita que múltiples workers escriban simultáneamente sobre los parámetros globales, reduciendo el riesgo de condiciones de carrera asociadas a actualizaciones concurrentes sobre memoria compartida.

Para proteger la comprobación y modificación de `processed[job_id]`, el modelo utiliza una sección atomic, de modo que la validación de que un trabajo no haya sido procesado previamente y su posterior marcado se ejecuten como una única operación lógica.

Un fragmento representativo es:

```
atomic {  
    assert(processed[job_id] == 0);  
    processed[job_id] = 1;  
}
```

El primer assert permite verificar que ningún trabajo sea ejecutado más de una vez.

6.2 Procesos y canales

El modelo se compone principalmente del proceso coordinador, representado mediante `init`, y del proceso Worker.

El proceso `init` cumple las funciones de coordinación: crea los workers, envía los trabajos, envía las señales de finalización, recibe los resultados parciales, comprueba que todos los trabajos hayan sido procesados y finalmente representa la actualización de los pesos.

El proceso “proctype Worker(byte worker_id)” representa a cada worker del Worker Pool. Cada instancia espera elementos del canal jobs. Cuando recibe un identificador de trabajo válido, verifica que el trabajo no haya sido procesado anteriormente, lo marca como procesado y envía su identificador al canal results, representando el gradiente parcial producido.

Se definieron dos canales principales:

```
chan jobs = [7] of { byte };
```

```
chan results = [4] of { byte };
```

El canal jobs transporta los identificadores de los trabajos desde el coordinador hacia los workers. El canal results transporta los resultados desde los workers hacia el coordinador.

Además, se utiliza “#define STOP 255” como señal de finalización. Una vez enviados todos los trabajos, el coordinador incorpora una señal STOP por cada worker. Cuando un worker recibe dicho valor, abandona su ciclo de procesamiento y termina su ejecución.

Para la verificación se empleó la siguiente configuración reducida:

```
#define NUM_WORKERS 3
```

```
#define NUM_JOBS 4
```

Esta cantidad no representa la configuración utilizada durante los benchmarks en Go. Su finalidad es permitir que SPIN explore de manera exhaustiva las diferentes intercalaciones posibles entre procesos sin necesidad de modelar los 2,103,432 registros utilizados en el entrenamiento real.

6.3 Condiciones verificadas

El modelo incorpora instrucciones *assert* para verificar las propiedades de seguridad que deben mantenerse durante cualquier intercalación válida entre los procesos. Estas condiciones permiten comprobar que los trabajos y resultados no se dupliquen, que todos los trabajos sean procesados y que la actualización global de los parámetros ocurra únicamente después de completar la fase de reducción.

Propiedad verificada	Condición utilizada
El identificador del trabajo debe ser válido	<i>assert(job_id < NUM_JOBS)</i>
Un trabajo no puede ser procesado más de una vez	<i>assert(processed[job_id] == 0)</i>
El identificador del resultado debe ser válido	<i>assert(result_id < NUM_JOBS)</i>
Un resultado no puede ser recibido más de una vez	<i>assert(received[result_id] == 0)</i>

Deben recibirse todos los resultados	<i>assert(result_count == NUM_JOBS)</i>
Todos los trabajos deben haber sido procesados	<i>assert(processed[i] == 1)</i>
Todos los resultados deben haber sido recibidos	<i>assert(received[i] == 1)</i>
La actualización global debe realizarse al final	<i>assert(weights_updated)</i>

Una condición especialmente relevante es `assert(result_count == NUM_JOBS)`, ya que impide considerar finalizada la fase de procesamiento mientras no se hayan recibido todos los resultados parciales. Posteriormente, se comprueba individualmente que cada trabajo haya sido procesado y recibido. Solo después de estas verificaciones se establece `weights_updated = 1`, representando la actualización de los parámetros globales del modelo. De este modo, dentro de la abstracción Promela, los workers no realizan actualizaciones concurrentes sobre los pesos compartidos.

6.4 Evidencia de ejecución en SPIN

Para observar el comportamiento dinámico del modelo se ejecutó una simulación mediante SPIN, permitiendo visualizar la creación de los procesos Worker, la distribución de trabajos y su consumo mediante el canal jobs.

El comando utilizado fue: *spin -p -g -l promela/regression_workers.pml*

Figura 3. Simulación de la lógica concurrente del Worker Pool mediante Promela.

```

seung720@MAYHUA: /mnt/c/Users/SeuNg720/Documents/Grupo5-ProgramacionConcurrente
seung720@MAYHUA:/mnt/c/Users/SeuNg720/Documents/Grupo5-ProgramacionConcurrente$ spin -p -g -l promela/regression_workers.pml | head -n 45
0: proc - (:root:) creates proc 0 (:init:)
1: proc 0 (:init::1) promela/regression_workers.pml:102 (state 1) [i = 0]
   :init:(0):i = 0
2: proc 0 (:init::1) promela/regression_workers.pml:114 (state 8) [.(goto)]
3: proc 0 (:init::1) promela/regression_workers.pml:105 (state 2) [(((i<3)))]
Starting Worker with pid 1
4: proc 0 (:init::1) creates proc 1 (Worker)
4: proc 0 (:init::1) promela/regression_workers.pml:107 (state 3) [(run Worker(i))]
5: proc 0 (:init::1) promela/regression_workers.pml:109 (state 4) [i = (i+1)]
   :init:(0):i = 1
6: proc 0 (:init::1) promela/regression_workers.pml:105 (state 2) [(((i<3)))]
Starting Worker with pid 2
7: proc 0 (:init::1) creates proc 2 (Worker)
7: proc 0 (:init::1) promela/regression_workers.pml:107 (state 3) [(run Worker(i))]
8: proc 0 (:init::1) promela/regression_workers.pml:109 (state 4) [i = (i+1)]
   :init:(0):i = 2
9: proc 0 (:init::1) promela/regression_workers.pml:105 (state 2) [(((i<3)))]
Starting Worker with pid 3
10: proc 0 (:init::1) creates proc 3 (Worker)
10: proc 0 (:init::1) promela/regression_workers.pml:107 (state 3) [(run Worker(i))]
11: proc 0 (:init::1) promela/regression_workers.pml:109 (state 4) [i = (i+1)]
   :init:(0):i = 3
12: proc 0 (:init::1) promela/regression_workers.pml:111 (state 5) [else]
13: proc 0 (:init::1) promela/regression_workers.pml:104 (state 9) [break]
14: proc 0 (:init::1) promela/regression_workers.pml:120 (state 11) [i = 0]
   queue 1 (jobs):
   :init:(0):i = 0
15: proc 0 (:init::1) promela/regression_workers.pml:137 (state 18) [.(goto)]
   queue 1 (jobs):
16: proc 0 (:init::1) promela/regression_workers.pml:123 (state 12) [(((i<4)))]
   queue 1 (jobs):
17: proc 0 (:init::1) promela/regression_workers.pml:125 (state 13) [jobs!i]
   queue 1 (jobs): [0]
18: proc 1 (Worker:1) promela/regression_workers.pml:59 (state 1) [jobs?job_id]
   queue 1 (jobs):
   Worker(1):job_id = 0
19: proc 0 (:init::1) promela/regression_workers.pml:127 (state 14) [i = (i+1)]
   queue 1 (jobs):
   :init:(0):i = 1
20: proc 1 (Worker:1) promela/regression_workers.pml:66 (state 4) [else]
   queue 1 (jobs):
21: proc 1 (Worker:1) promela/regression_workers.pml:68 (state 5) [assert((job_id<4))]
   queue 1 (jobs):
22: proc 1 (Worker:1) promela/regression_workers.pml:75 (state 6) [assert((processed[job_id]==0))]
   queue 1 (jobs):
seung720@MAYHUA:/mnt/c/Users/SeuNg720/Documents/Grupo5-ProgramacionConcurrente$

```

Se observa la creación de los procesos Worker y la comunicación de trabajos mediante el canal jobs.

Durante el procesamiento, cada Worker recibe un identificador de trabajo y verifica que este no haya sido atendido previamente. Una vez finalizado el cálculo representado por el trabajo, el resultado es comunicado al coordinador. Al concluir todos los trabajos, el proceso init verifica que cada uno haya sido procesado y que su resultado correspondiente haya sido recibido antes de realizar la actualización de los pesos.

Figura 4. Finalización de la simulación del modelo Promela.

```

seung720@MAYHUA: /mnt/c/Users/SeuNg720/Documents/Grupo5-ProgramacionConcurrente
seung720@MAYHUA: /mnt/c/Users/SeuNg720/Documents/Grupo5-ProgramacionConcurrente$ spin -p -g -l promela/regression_workers.pml | tail -n 60
137:  proc  0 (:init::1) promela/regression_workers.pml:202 (state 46)      [assert((received[i]==1))]
      queue 1 (jobs):
      queue 2 (results):
138:  proc  0 (:init::1) promela/regression_workers.pml:204 (state 47)      [i = (i+1)]
      queue 1 (jobs):
      queue 2 (results):
      :init:(0):i = 2
139:  proc  0 (:init::1) promela/regression_workers.pml:218 (state 51)      [.(goto)]
      queue 1 (jobs):
      queue 2 (results):
140:  proc  0 (:init::1) promela/regression_workers.pml:199 (state 44)      [[[i<4]]]
      queue 1 (jobs):
      queue 2 (results):
141:  proc  0 (:init::1) promela/regression_workers.pml:201 (state 45)      [assert((processed[i]==1))]
      queue 1 (jobs):
      queue 2 (results):
142:  proc  0 (:init::1) promela/regression_workers.pml:202 (state 46)      [assert((received[i]==1))]
      queue 1 (jobs):
      queue 2 (results):
143:  proc  0 (:init::1) promela/regression_workers.pml:204 (state 47)      [i = (i+1)]
      queue 1 (jobs):
      queue 2 (results):
      :init:(0):i = 3
144:  proc  0 (:init::1) promela/regression_workers.pml:218 (state 51)      [.(goto)]
      queue 1 (jobs):
      queue 2 (results):
145:  proc  0 (:init::1) promela/regression_workers.pml:199 (state 44)      [[[i<4]]]
      queue 1 (jobs):
      queue 2 (results):
146:  proc  0 (:init::1) promela/regression_workers.pml:201 (state 45)      [assert((processed[i]==1))]
      queue 1 (jobs):
      queue 2 (results):
147:  proc  0 (:init::1) promela/regression_workers.pml:202 (state 46)      [assert((received[i]==1))]
      queue 1 (jobs):
      queue 2 (results):
148:  proc  0 (:init::1) promela/regression_workers.pml:204 (state 47)      [i = (i+1)]
      queue 1 (jobs):
      queue 2 (results):
      :init:(0):i = 4
149:  proc  0 (:init::1) promela/regression_workers.pml:218 (state 51)      [.(goto)]
      queue 1 (jobs):
      queue 2 (results):
150:  proc  0 (:init::1) promela/regression_workers.pml:206 (state 48)      [else]
      queue 1 (jobs):
      queue 2 (results):
151:  proc  0 (:init::1) promela/regression_workers.pml:207 (state 49)      [goto :b5]
      queue 1 (jobs):
      queue 2 (results):
152:  proc  0 (:init::1) promela/regression_workers.pml:218 (state 53)      [weights_updated = 1]
      queue 1 (jobs):
      queue 2 (results):
      weights_updated = 1
153:  proc  0 (:init::1) promela/regression_workers.pml:221 (state 54)      [assert(weights_updated)]
      queue 1 (jobs):
      queue 2 (results):
154:  proc  0 (:init::1) promela/regression_workers.pml:222 (state 55)      [assert((result_count==4))]
      queue 1 (jobs):
      queue 2 (results):
154:  proc  0 (:init::1) terminates
4 processes created
seung720@MAYHUA: /mnt/c/Users/SeuNg720/Documents/Grupo5-ProgramacionConcurrente$

```

Se verifica que los trabajos fueron procesados y sus resultados recibidos antes de ejecutar la actualización global de los pesos.

La simulación anterior permite visualizar una posible intercalación entre los procesos, pero no verifica por sí sola todas las ejecuciones posibles. Por ello, se generó el verificador de SPIN y se realizó una exploración exhaustiva del espacio de estados mediante los siguientes comandos:

```

spin -a promela/regression_workers.pml
gcc -O2 -o pan pan.c
./pan

```

Figura 5. Verificación exhaustiva del modelo concurrente mediante SPIN.

```

seung720@MAYHUA:/mnt/c/Users/SeuNg720/Documents/Grupo5-ProgramacionConcurrente$ ./pan

(Spin Version 6.5.2 -- 6 December 2019)
+ Partial Order Reduction

Full statespace search for:
  never claim           - (none specified)
  assertion violations   +
  acceptance cycles     - (not selected)
  invalid end states    +

State-vector 80 byte, depth reached 120, errors: 0
 143799 states, stored
  97465 states, matched
 241264 transitions (= stored+matched)
   12 atomic steps
hash conflicts:      265 (resolved)

Stats on memory usage (in Megabytes):
 14.811    equivalent memory usage for states (stored*(State-vector + overhead))
 10.054    actual memory usage for states (compression: 67.88%)
           state-vector as stored = 45 byte + 28 byte overhead
128.000    memory used for hash table (-w24)
  0.534    memory used for DFS stack (-m10000)
138.495    total actual memory usage

unreached in proctype Worker
  (0 of 15 states)
unreached in init
  (0 of 56 states)

pan: elapsed time 0.04 seconds
pan: rate 3594975 states/second

```

La exploración alcanzó 143,799 estados y 241,264 transiciones sin detectar violaciones de las propiedades definidas ni estados finales inválidos (errors: 0).

La verificación produjo los siguientes resultados:

Indicador	Resultado
Profundidad alcanzada	120
Estados almacenados	143,799
Estados coincidentes	97,465
Transiciones exploradas	241,264
Pasos atómicos	12
Errores detectados	0
Estados no alcanzados en Worker	0 de 15
Estados no alcanzados en init	0 de 56

El resultado errors: 0 indica que, dentro del espacio de estados explorado para la abstracción definida, SPIN no encontró violaciones de los assert establecidos ni estados finales inválidos. En consecuencia, el modelo validó la lógica de coordinación planteada para el Worker Pool: cada trabajo es procesado una única vez, cada resultado es recolectado una única vez y la actualización de los pesos ocurre únicamente después de reunir todos los resultados parciales.

Adicionalmente, el modelo se amplió con tres propiedades en lógica temporal lineal (LTL): `safe_update` ($\square$ (updating $\rightarrow$ result_count == NUM_JOBS)), que exige que la actualización de pesos solo ocurra después de recibir todos los resultados; `termination` ($\diamond$ done), que exige que el sistema siempre termine; y `mutex` ($\square$ (in_update $\leq$ 1)), que exige que a lo sumo un proceso esté en la sección de actualización de pesos. Se verificaron con SPIN 6.5.2 para distintas combinaciones de workers y jobs (resultados completos en results/promela/ del repositorio):

Workers / jobs	Corrida	Estados almacenados	Errores
2 / 4	Seguridad + 3 propiedades LTL	12,358	0
3 / 4	Seguridad + 3 propiedades LTL	145,563	0
4 / 4	Seguridad + 3 propiedades LTL	1,080,202	0

Tabla. Verificación de seguridad y propiedades LTL en SPIN 6.5.2.

En las tres variantes SPIN no encontró violaciones de aserciones, estados finales inválidos ni contraejemplos de las propiedades LTL. La propiedad mutex demuestra formalmente que la exclusión mutua sobre los pesos está garantizada por el diseño, lo que justifica no usar sync.Mutex en la implementación en Go. La variante de 4 workers y 8 jobs superó los 34 millones de estados y 3.2 GB de memoria sin terminar, un ejemplo de la explosión del espacio de estados; por ello la verificación exhaustiva se realiza con configuraciones pequeñas, suficientes para cubrir todas las intercalaciones de la lógica de sincronización.

7. Implementación en Go

La solución fue implementada íntegramente en Go y organizada en componentes separados para la preparación de los datos, representación del modelo de Regresión Lineal, entrenamiento secuencial, entrenamiento concurrente y medición de rendimiento. El archivo limpio SPARCS_2022_clean_go.csv, obtenido en la etapa descrita en la sección 5, constituye la entrada de esta implementación.

La arquitectura desarrollada mantiene una misma representación del modelo y los mismos datos de entrenamiento para las versiones secuencial y concurrente. De esta manera, la comparación de rendimiento se realiza modificando únicamente la estrategia empleada para calcular los gradientes durante el entrenamiento.

Los principales archivos involucrados son:

Archivo	Responsabilidad principal
dataset.go	Lectura y representación inicial del dataset
preprocessing.go	Identificación de categorías y codificación de las variables

scaling.go	Cálculo y aplicación de los parámetros de escalamiento
regression.go	Representación y operaciones comunes del modelo de Regresión Lineal
sequential.go	Entrenamiento secuencial
concurrent.go	Entrenamiento concurrente
benchmark.go	Medición de tiempos, speedup y eficiencia
resource_profile.go	Medición de memoria y asignaciones
cpu_profile.go	Ejecución destinada a observar el aprovechamiento de CPU

Los últimos tres archivos se describen con mayor detalle en las secciones correspondientes a metodología experimental y análisis de recursos.

7.1 Preparación de datos para Regresión Lineal

El dataset limpio contiene 2,103,432 registros y 11 columnas. Sin embargo, la Regresión Lineal requiere que las variables de entrada se encuentren representadas numéricamente. Por ello, antes del entrenamiento se realizó una etapa adicional de preprocesamiento que incluyó identificación de categorías, codificación, partición Train/Test y escalamiento.

Las variables numéricas y binarias se conservaron directamente:

- Length of Stay
- LOS_120_plus
- APR Severity of Illness Code
- Severity_unknown

Por otro lado, las variables categóricas fueron transformadas mediante codificación one-hot. Para evitar columnas redundantes, se utilizó una categoría como referencia en cada variable categórica.

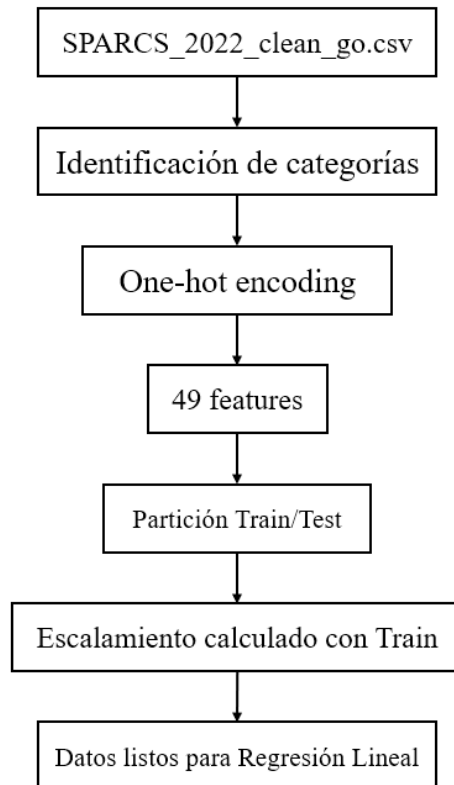
La configuración obtenida fue la siguiente:

Variable	Categorías encontradas	Features generadas
Variables numéricas/binarias	-	4
Age Group	5	4
Type of Admission	6	5
APR MDC Description	26	25
APR Medical Surgical Description	3	2

Payment Typology 1	9	8
Emergency Department Indicator	2	1
Total		49

La variable Total Costs no se incluye en estas 49 características, debido a que constituye la variable objetivo que el modelo debe estimar.

Figura 6. Flujo de preparación de datos para la Regresión Lineal



El dataset limpio es transformado mediante codificación de variables categóricas, partición Train/Test y escalamiento antes de ser utilizado por las implementaciones secuencial y concurrente.

Después de la codificación, los 2,103,432 registros fueron divididos de forma reproducible en conjuntos de entrenamiento y prueba:

Conjunto	Registros	Proporción
Train	1,682,537	79.99 %
Test	420,895	20.01 %
Total	2,103,432	100 %

El escalamiento se calculó utilizando únicamente los registros pertenecientes a Train, evitando que información estadística del conjunto Test intervenga en la preparación del modelo. Para una variable x , la transformación utilizada corresponde a:

$$z = \frac{x - \mu_{Train}}{\sigma_{Train}}$$

Los parámetros obtenidos fueron:

Variable	Media Train	Desviación estándar Train
----------	-------------	---------------------------

Length of Stay	5.8309	8.8607
APR Severity of Illness Code	2.1462	0.9626
Total Costs	23,423.18	49,952.01

Las características binarias y provenientes de one-hot encoding permanecen representadas mediante valores 0 y 1. La variable objetivo fue estandarizada durante el entrenamiento, mientras que las métricas finales se reportan nuevamente en su escala monetaria original.

De esta forma, tanto la implementación secuencial como la concurrente reciben exactamente el mismo conjunto de entrenamiento de 1,682,537 registros, cada uno representado mediante 49 características numéricas.

7.2 Implementación secuencial

La implementación secuencial constituye la línea base utilizada para medir posteriormente el beneficio de la concurrencia. El entrenamiento se realiza mediante Batch Gradient Descent, procesando secuencialmente todos los registros del conjunto Train en cada época.

Para un registro i , la predicción del modelo se calcula mediante:

$$\hat{y}_i = b + \sum_{j=1}^m w_j x_{ij}$$

donde $m = 49$ corresponde al número de características, w_j representa los pesos del modelo y b el término independiente.

A partir de la diferencia entre la predicción y el valor real se calcula el error:

$$e_i = \hat{y}_i - y_i$$

Durante cada época, la implementación recorre los 1,682,537 registros y acumula las contribuciones al gradiente. Una vez procesado todo el conjunto de entrenamiento, el gradiente promedio se utiliza para actualizar los parámetros del modelo.

Conceptualmente, el procedimiento corresponde a:

Inicializar pesos

Para cada época:

 Inicializar gradiente acumulado

 Para cada registro de Train:

 Calcular predicción

Calcular error
Acumular gradiente

Promediar gradiente
Actualizar pesos

En la ejecución experimental se utilizaron:

Épocas: 100

Learning rate: 0.01

Registros: 1,682,537

Features: 49

La versión secuencial no utiliza goroutines para distribuir el cálculo del gradiente. Por ello, constituye una referencia apropiada para comparar posteriormente los tiempos obtenidos por el modelo concurrente.

7.3 Implementación concurrente

La implementación concurrente conserva el mismo modelo matemático y los mismos parámetros de entrenamiento utilizados por la versión secuencial. La diferencia se encuentra en la forma en que se calcula el gradiente de cada época.

El conjunto Train está dividido en múltiples trabajos o jobs. Cada trabajo contiene una porción independiente de los registros y puede ser procesado simultáneamente por uno de los workers disponibles.

Cada worker calcula sobre su partición:

- la suma parcial de los gradientes correspondientes a los pesos;
- la contribución parcial al gradiente del término independiente;
- la información necesaria para posteriormente combinar los resultados.

Los workers no actualizan directamente los pesos del modelo. En su lugar, envían sus resultados parciales al proceso coordinador. Una vez recibidos todos los resultados, estos son reducidos para obtener el mismo gradiente global que se calcularía recorriendo secuencialmente el conjunto completo.

Si el conjunto de entrenamiento se divide en K trabajos, el gradiente global puede expresarse conceptualmente como:

$$G = \sum_{k=1}^K G_k$$

donde G_k representa el gradiente parcial calculado para el trabajo k .

La actualización de los parámetros se realiza únicamente después de completar esta reducción:

$$w_{t+1} = w_t - \eta \frac{G}{N}$$

donde η corresponde al learning rate y N al número total de registros procesados.

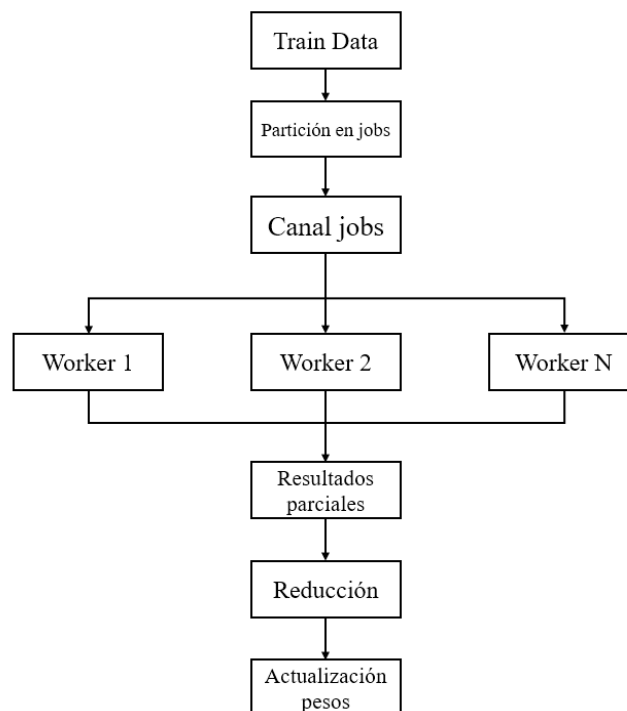
Este diseño permite paralelizar la sección computacionalmente más costosa del entrenamiento sin modificar la formulación matemática del modelo.

Los experimentos confirmaron posteriormente esta equivalencia: la diferencia máxima entre los parámetros obtenidos por ambas implementaciones fue 0.000000000000, y las métricas sobre Test resultaron idénticas.

7.4 Patrón Worker Pool

Para gestionar el procesamiento concurrente se utilizó el patrón Worker Pool. En lugar de crear una goroutine nueva para cada registro del dataset, se mantiene una cantidad limitada de workers que consumen trabajos desde un canal compartido.

Figura 7. Arquitectura de la implementación concurrente en Go mediante el patrón Worker Pool



Los jobs son distribuidos entre un conjunto limitado de workers, los cuales calculan gradientes parciales y envían sus resultados para su posterior reducción y actualización de los parámetros del modelo.

Cada worker permanece disponible para recibir trabajos hasta que el canal correspondiente es cerrado o se completa el procesamiento asignado. Esto evita crear millones de goroutines y permite controlar explícitamente el grado de paralelismo.

Durante la ejecución principal sobre el equipo utilizado para las pruebas se emplearon 12 workers, correspondientes a los 12 procesadores lógicos disponibles. En esa configuración se generaron 48 jobs por época.

Para estudiar la escalabilidad de la solución, posteriormente el mismo algoritmo fue evaluado utilizando configuraciones de: 1, 2, 4, 8, 12, 16, 24 y 32 workers

Estas configuraciones y sus resultados se desarrollan en las secciones 8, 9 y 10.

7.5 Mecanismos de sincronización utilizados

La implementación concurrente utiliza principalmente tres mecanismos proporcionados por Go: goroutines, channels y `sync.WaitGroup`.

Las goroutines representan los workers encargados de ejecutar de forma concurrente el cálculo de los gradientes parciales. Los channels permiten transferir trabajos y resultados entre el coordinador y los workers sin que estos tengan que modificar directamente las mismas estructuras globales.

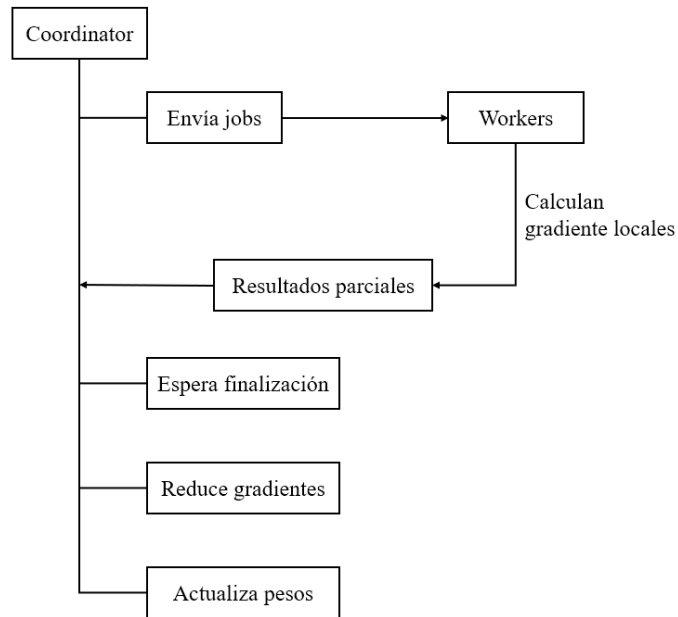
`sync.WaitGroup` se utiliza para coordinar la finalización de las goroutines y garantizar que el programa no continúe con una fase que dependa de resultados todavía no calculados.

Una característica importante del diseño es que los workers no actualizan directamente los pesos globales del modelo. Cada worker opera sobre variables locales y produce un gradiente parcial. La reducción de dichos gradientes y la actualización de los pesos se realizan posteriormente desde un único flujo de ejecución.

Por esta razón, no fue necesario utilizar un `sync.Mutex` para proteger los pesos durante el cálculo de los gradientes. Esta decisión reduce la contención que producirían múltiples workers intentando adquirir continuamente un mismo bloqueo.

La sincronización de la implementación puede resumirse conceptualmente de la siguiente manera:

Figura 8. Flujo de sincronización de la implementación concurrente en Go



El coordinador distribuye los jobs entre los workers, recibe los gradientes parciales, espera la finalización de las goroutines, reduce los resultados y posteriormente actualiza los pesos del modelo.

Este mecanismo coincide con la abstracción validada previamente mediante Promela: los cálculos parciales pueden ejecutarse concurrentemente, mientras que la actualización del estado global del modelo se realiza únicamente después de reunir los resultados requeridos.

8. Metodología de medición

Con el propósito de comparar de manera consistente el rendimiento de las implementaciones secuencial y concurrente, se definió una metodología experimental común para todas las configuraciones evaluadas. Las mediciones se realizaron manteniendo constantes el conjunto de datos, el preprocesamiento, los parámetros del modelo y el número de épocas, modificándose únicamente la cantidad de workers utilizada en la versión concurrente.

Para los resultados oficiales se utilizaron únicamente ejecuciones realizadas en un mismo equipo y bajo la misma configuración de software. De esta manera, se evitó combinar tiempos obtenidos en computadoras con características de hardware diferentes.

La comparación se centra principalmente en el tiempo de entrenamiento, el speedup y la eficiencia alcanzada al aumentar la cantidad de workers. Las métricas predictivas del modelo también fueron verificadas para comprobar que la paralelización no modificara el resultado matemático del entrenamiento.

8.1 Entorno de ejecución

Los benchmarks oficiales fueron realizados sobre un equipo con las siguientes características:

Componente	Especificación
Procesador	AMD Ryzen 5 9600X
Núcleos físicos	6
Procesadores lógicos	12
Memoria RAM	15.6 GB
Sistema operativo	Microsoft Windows 11 Pro, 64 bits
Versión de Go	Go 1.27.0
Registros totales	2,103,432
Registros de entrenamiento	1,682,537
Features	49
Épocas	100
Learning rate	0.01

La cantidad de procesadores lógicos fue obtenida desde el propio sistema y coincide con los 12 procesadores lógicos detectados por el runtime de Go durante la ejecución.

Se mantuvieron constantes las 100 épocas, el learning rate de 0.01, el conjunto Train y las 49 características utilizadas por el modelo. Esto permite que las diferencias de tiempo observadas entre las configuraciones se atribuyan principalmente a la estrategia de ejecución y al grado de concurrencia empleado.

8.2 Número de ejecuciones

Para reducir el efecto de variaciones ocasionales en los tiempos de ejecución, cada configuración fue evaluada mediante siete ejecuciones.

Se realizaron siete mediciones para la versión secuencial y siete para cada una de las configuraciones concurrentes. En todas las ejecuciones se conservaron los mismos parámetros del modelo y el mismo conjunto de entrenamiento.

Por tanto, la metodología utilizada fue:

Tipo de ejecución	Cantidad de mediciones
Secuencial	7
Concurrente con 1 worker	7
Concurrente con 2 workers	7
Concurrente con 4 workers	7
Concurrente con 8 workers	7
Concurrente con 12 workers	7
Concurrente con 16 workers	7
Concurrente con 24 workers	7
Concurrente con 32 workers	7

En total, el benchmark formal comprende 63 ejecuciones de entrenamiento:

$$9 \text{ configuraciones} \times 7 \text{ ejecuciones} = 63 \text{ ejecuciones}$$

Las ejecuciones mostradas previamente como comparación inicial entre secuencial y concurrente fueron utilizadas para validar el funcionamiento del programa, pero los resultados oficiales de rendimiento se obtienen del benchmark formal de siete ejecuciones por configuración.

8.3 Configuraciones de workers

La implementación concurrente fue evaluada modificando progresivamente la cantidad de workers disponibles: 1, 2, 4, 8, 12, 16, 24, 32

La inclusión de diferentes cantidades de workers permite estudiar cómo evoluciona el rendimiento conforme aumenta el grado de concurrencia.

Las configuraciones de 1, 2, 4 y 8 workers permiten observar el comportamiento previo a alcanzar la totalidad de los 12 procesadores lógicos disponibles. La configuración de 12 workers coincide directamente con la capacidad lógica reportada por el procesador utilizado.

Adicionalmente, se probaron 16, 24 y 32 workers. Estas configuraciones superan el número de procesadores lógicos y permiten analizar el efecto de la sobresuscripción, es decir, la existencia de más workers concurrentes que unidades lógicas de ejecución disponibles.

Esta evaluación permite identificar tanto la región donde aumentar workers mejora el rendimiento como el punto a partir del cual aparecen rendimientos decrecientes debido a costos de coordinación, planificación y gestión de las goroutines.

Configuración	Relación con el hardware
1 worker	Línea base concurrente
2 workers	Paralelismo bajo
4 workers	Paralelismo intermedio
8 workers	Próximo a la capacidad lógica
12 workers	Igual al número de procesadores lógicos
16 workers	Sobresuscripción
24 workers	2 x procesadores lógicos
32 workers	Sobresuscripción alta

8.4 Cálculo de media recortada

Para representar el tiempo de cada configuración no se utilizó directamente una única ejecución ni el promedio simple de las siete mediciones. En su lugar, se calculó una media recortada, reduciendo la influencia de mediciones excepcionalmente rápidas o lentas.

Para cada configuración se siguió el siguiente procedimiento:

1. Se registraron los siete tiempos de ejecución.
2. Los tiempos se ordenaron de menor a mayor.
3. Se descartó el valor mínimo.
4. Se descartó el valor máximo.
5. Se calculó la media aritmética de las cinco mediciones restantes.

Si los tiempos ordenados de una configuración se representan como:

$$t_1 \leq t_2 \leq t_3 \leq t_4 \leq t_5 \leq t_6 \leq t_7$$

la media recortada utilizada corresponde a:

$$T_{recortada} = \frac{t_2 \leq t_3 \leq t_4 \leq t_5 \leq t_6}{5}$$

Este valor se utilizó posteriormente como tiempo representativo de cada configuración para calcular el speedup y analizar la escalabilidad.

El speedup se determina mediante:

$$Speedup = \frac{T_{secuencial}}{T_{concurrente}}$$

donde $T_{secuencial}$ corresponde a la media recortada de la implementación secuencial y $T_{concurrente}$ a la media recortada de cada configuración concurrente.

Asimismo, la eficiencia respecto al número de workers se calcula mediante:

$$Eficiencia = \frac{Speedup}{Número\ de\ workers}$$

Estas métricas permiten distinguir entre una reducción absoluta del tiempo de ejecución y el nivel de aprovechamiento efectivo de los recursos concurrentes utilizados.

9. Resultados experimentales

A partir de la metodología descrita en la sección anterior, se ejecutó el benchmark formal para la implementación secuencial y para las configuraciones concurrentes de 1, 2, 4, 8, 12, 16, 24 y 32 workers. Para cada configuración se realizaron siete ejecuciones y se utilizó la media recortada como tiempo representativo, descartando previamente la medición mínima y máxima.

Los resultados permiten comparar el comportamiento temporal de ambas implementaciones y verificar si el aumento del grado de concurrencia produce una reducción efectiva del tiempo de entrenamiento.

9.1 Tiempos de ejecución

La implementación secuencial obtuvo una media recortada de 1.8694 segundos. Por su parte, el tiempo de la implementación concurrente disminuyó progresivamente al aumentar la cantidad de workers hasta alcanzar valores cercanos a 0.38 segundos.

Configuración	Media recortada
Secuencial	1869.45 ms
1 worker	1820.02 ms
2 workers	951.15 ms
4 workers	510.57 ms
8 workers	397.33 ms
12 workers	387.66 ms

16 workers	382.95 ms
24 workers	384.63 ms
32 workers	384.12 ms

Los resultados muestran una reducción considerable del tiempo de entrenamiento al pasar de la ejecución secuencial a configuraciones con varios workers. La disminución es especialmente pronunciada entre 1 y 8 workers. El menor tiempo representativo registrado corresponde a la configuración de 16 workers, con una media recortada de 382.95 ms. A partir de esa configuración, los tiempos permanecen alrededor de 0.38 segundos, sin observarse reducciones relevantes adicionales.

9.2 Speedup y eficiencia

Tomando como referencia la media recortada de la implementación secuencial, se calculó el speedup de cada configuración concurrente mediante la relación definida en la sección 8.4. Asimismo, se calculó la eficiencia dividiendo el speedup entre el número de workers empleados.

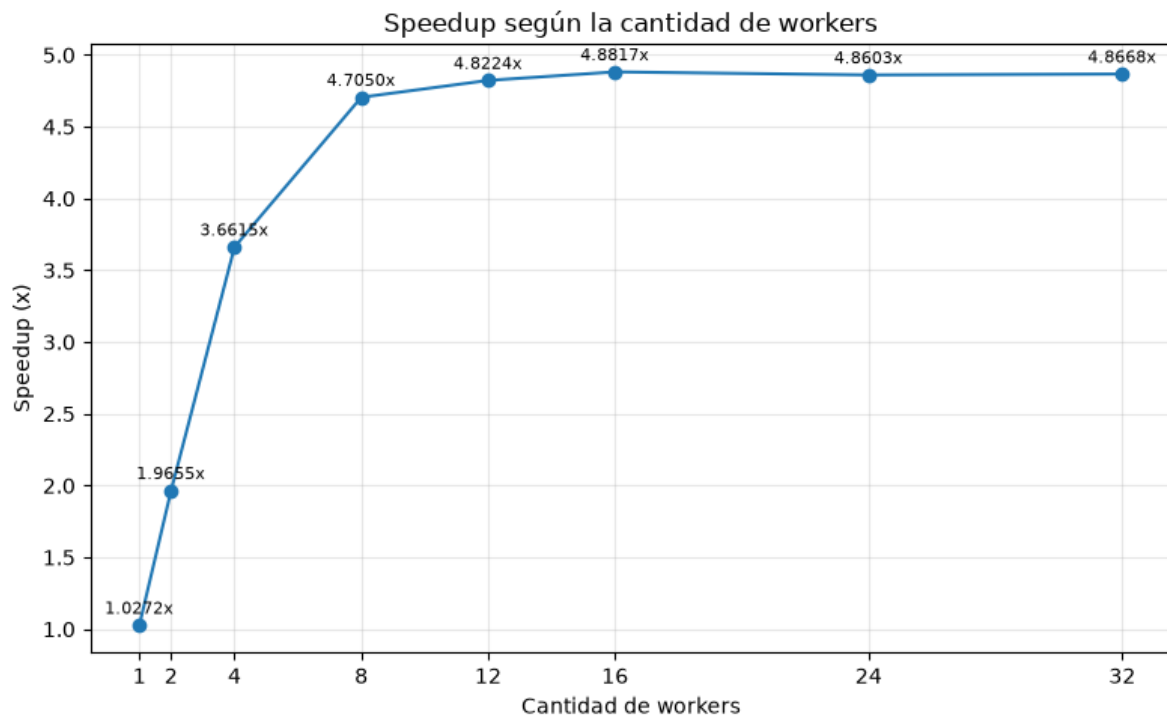
Los resultados obtenidos se resumen en la siguiente tabla:

Configuración	Tiempo	Speedup	Eficiencia
Secuencial	1869.45 ms	1.0000x	-
1 worker	1820.02 ms	1.0272x	1.0272
2 workers	951.15 ms	1.9655x	0.9827
4 workers	510.57 ms	3.6615x	0.9154
8 workers	397.33 ms	4.7050x	0.5881
12 workers	387.66 ms	4.8224x	0.4019
16 workers	382.95 ms	4.8817x	0.3051
24 workers	384.63 ms	4.8603x	0.2025
32 workers	384.12 ms	4.8668x	0.1521

El speedup aumenta de manera marcada en las primeras configuraciones concurrentes. Con 2 workers se obtuvo un speedup de 1.9655x, mientras que con 4 workers este valor ascendió a 3.6615x. Con 8 workers se alcanzó un speedup de 4.7050x y el mayor valor observado fue de 4.8817x con 16 workers.

La eficiencia, en cambio, disminuye conforme aumenta el número de workers. Con 2 workers se obtuvo una eficiencia de 0.9827 y con 4 workers de 0.9154. A partir de 8 workers esta métrica desciende progresivamente, llegando a 0.1521 con 32 workers.

Figura 9. Speedup según la cantidad de workers



La Figura 9 muestra un incremento pronunciado del speedup durante las primeras configuraciones concurrentes. El valor aumenta de 1.0272x con 1 worker a 4.7050x con 8 workers y alcanza su máximo observado de 4.8817x con 16 workers. A partir de este punto, el speedup se mantiene cercano a 4.86x, evidenciando una estabilización de la mejora obtenida al incrementar la cantidad de workers.

9.3 Comparación secuencial vs. concurrente

La versión secuencial y la versión concurrente utilizaron exactamente el mismo conjunto Train, las mismas 49 características, 100 épocas y un learning rate de 0.01. Por tanto, la diferencia entre ambas implementaciones corresponde a la estrategia utilizada para calcular el gradiente.

La implementación secuencial obtuvo una media recortada de 1869.45 ms, mientras que la configuración concurrente de 12 workers redujo este tiempo a 387.66 ms y la configuración de 16 workers alcanzó el menor tiempo observado, con 382.95 ms.

Indicador	Secuencial	Concurrente 12 workers	Concurrente 16 workers
Media recortada	1869.45 ms	387.66 ms	382.95 ms

Speedup	1.0000x	4.8224x	4.8817x
Workers	-	12	16
Épocas	100	100	100
Features	49	49	49

En términos absolutos, la configuración de 16 workers presentó el menor tiempo de ejecución. No obstante, la diferencia temporal con respecto a 12 workers fue reducida: aproximadamente 4.71 ms. Esta proximidad entre ambas configuraciones será analizada posteriormente junto con la utilización de recursos y la eficiencia para determinar el punto de equilibrio de la solución.

9.4 Métricas del modelo

La comparación de las métricas del modelo permite verificar que la estrategia concurrente no alteró el resultado matemático obtenido por la implementación secuencial.

La evaluación sobre el conjunto Test produjo las siguientes métricas tanto para la versión secuencial como para la concurrente:

Métrica	Secuencial	Concurrente
MSE	1,337,138,841.47	1,337,138,841.47
RMSE	\$36,566.91	\$36,566.91
MAE	\$12,216.60	\$12,216.60
R^2	0.467884	0.467884

Las métricas predictivas obtenidas por las implementaciones secuencial y concurrente fueron idénticas. Ambas alcanzaron un MSE de 1,337,138,841.47, un RMSE de \$36,566.91, un MAE de \$12,216.60 y un coeficiente de determinación R^2 de 0.467884.

Adicionalmente, la comparación de los parámetros obtenidos al finalizar el entrenamiento mostró una diferencia máxima de 0.000000000000 entre ambas implementaciones. Esto evidencia que la estrategia concurrente utilizada conserva el resultado numérico producido por el algoritmo secuencial y que la mejora observada corresponde al tiempo de procesamiento, no a una modificación del modelo matemático.

Figura 10. Evidencia de ejecución de las implementaciones secuencial y concurrente en Go.

ENTRENAMIENTO SECUENCIAL	ENTRENAMIENTO CONCURRENTE
Épocas: 100 Learning rate: 0.0100 Registros: 1682537 Features: 49 Época 1/100 MSE estandarizado: 1.000000 Época 10/100 MSE estandarizado: 0.837608 Época 20/100 MSE estandarizado: 0.724230 Época 30/100 MSE estandarizado: 0.653482 Época 40/100 MSE estandarizado: 0.608326 Época 50/100 MSE estandarizado: 0.578791 Época 60/100 MSE estandarizado: 0.558971 Época 70/100 MSE estandarizado: 0.545320 Época 80/100 MSE estandarizado: 0.535675 Época 90/100 MSE estandarizado: 0.528691 Época 100/100 MSE estandarizado: 0.523518 Entrenamiento secuencial completado. Tiempo de entrenamiento: 1.9242788s Evaluando modelo sobre Test... RESULTADOS SECUENCIALES - TEST MSE: 1337138841.47 RMSE: \$36566.91 MAE: \$12216.60 R²: 0.467884	Épocas: 100 Learning rate: 0.0100 Registros: 1682537 Features: 49 Workers: 12 Jobs/época: 48 Época 1/100 MSE estandarizado: 1.000000 Época 10/100 MSE estandarizado: 0.837608 Época 20/100 MSE estandarizado: 0.724230 Época 30/100 MSE estandarizado: 0.653482 Época 40/100 MSE estandarizado: 0.608326 Época 50/100 MSE estandarizado: 0.578791 Época 60/100 MSE estandarizado: 0.558971 Época 70/100 MSE estandarizado: 0.545320 Época 80/100 MSE estandarizado: 0.535675 Época 90/100 MSE estandarizado: 0.528691 Época 100/100 MSE estandarizado: 0.523518 Entrenamiento concurrente completado. Tiempo de entrenamiento: 419.5301ms RESULTADOS CONCURRENTES - TEST MSE: 1337138841.47 RMSE: \$36566.91 MAE: \$12216.60 R²: 0.467884

10. Análisis de escalabilidad y trade-offs

Los resultados experimentales evidencian que la implementación concurrente mejora de forma significativa el tiempo de entrenamiento respecto a la versión secuencial. Sin embargo, esta mejora no crece proporcionalmente con la cantidad de workers. Mientras que las primeras configuraciones muestran incrementos importantes de speedup, a partir de cierto grado de concurrencia aparecen rendimientos decrecientes.

La Figura 9 permite observar este comportamiento: el speedup aumenta rápidamente durante las primeras configuraciones y posteriormente tiende a estabilizarse alrededor de 4.8x. Por tanto, el incremento del número de workers continúa aumentando el grado de concurrencia del programa, pero deja de traducirse en reducciones proporcionales del tiempo de ejecución.

10.1 Comportamiento de la escalabilidad

Entre 1 y 4 workers se observa una escalabilidad cercana a la relación esperada respecto al número de workers. Con 2 workers se obtuvo un speedup de 1.9655x y una eficiencia de 0.9827, mientras que con 4 workers se alcanzó un speedup de 3.6615x y una eficiencia de 0.9154.

Estos resultados indican que, en las primeras configuraciones, una parte importante del trabajo computacional puede distribuirse eficazmente entre los workers disponibles.

Con 8 workers el speedup aumentó hasta 4.7050x; sin embargo, la eficiencia descendió a 0.5881. Posteriormente, con 12 workers se obtuvo un speedup de 4.8224x y con 16 workers un máximo observado de 4.8817x. A partir de esta región, aumentar la cantidad de workers produjo mejoras muy reducidas o inexistentes.

Las configuraciones de 24 y 32 workers obtuvieron speedups de 4.8603x y 4.8668x respectivamente, valores ligeramente inferiores al obtenido con 16 workers. Esto muestra que la solución alcanza una región de saturación aproximadamente entre 12 y 16 workers, después de la cual incrementar el grado de concurrencia no produce una mejora adicional significativa.

Esto coincide además con las características del hardware utilizado, que dispone de 12 procesadores lógicos. Las configuraciones de 16, 24 y 32 workers superan dicha cantidad y, por tanto, corresponden a escenarios de sobresuscripción.

10.2 Rendimientos decrecientes y overhead

El comportamiento observado puede explicarse por la existencia de costos inherentes a la ejecución concurrente. En el diseño implementado, el trabajo debe dividirse en jobs, distribuirse entre los workers mediante channels, procesarse de forma concurrente, enviarse nuevamente al coordinador y finalmente reducir los gradientes parciales antes de actualizar los parámetros del modelo.

Por tanto, aumentar el número de workers no incrementa únicamente la capacidad de cálculo, sino también las tareas de coordinación asociadas a la concurrencia.

La implementación utiliza goroutines, channels y sync.WaitGroup para distribuir el trabajo y coordinar su finalización. Los workers calculan sus gradientes de manera local y la reducción se realiza posteriormente desde un único flujo de ejecución.

Este diseño evita que múltiples workers actualicen simultáneamente los pesos y, por ello, no requiere un sync.Mutex para protegerlos durante el cálculo. Esta decisión reduce la contención asociada a bloqueos sobre una estructura global compartida.

Sin embargo, incluso sin utilizar un mutex para los pesos, permanecen otros costos de concurrencia, como la comunicación mediante channels, la gestión y planificación de goroutines, la coordinación de su finalización y la reducción de resultados parciales.

Cuando existen suficientes recursos de procesamiento disponibles, estos costos son pequeños frente al beneficio de dividir el cálculo. En cambio, cuando la cantidad de workers supera la capacidad lógica del procesador, el beneficio adicional disminuye y el overhead representa una proporción mayor del tiempo total.

10.3 Trade-offs de la solución concurrente

Los resultados muestran que existe un compromiso entre tiempo de ejecución, eficiencia y grado de concurrencia.

Utilizar pocos workers mantiene una eficiencia elevada, pero no aprovecha completamente la capacidad de procesamiento disponible. Por ejemplo, con 2 workers la eficiencia fue de 0.9827, pero el speedup alcanzado fue de 1.9655x.

Al incrementar el número de workers se obtiene un menor tiempo de ejecución, pero la eficiencia respecto al número de workers comienza a disminuir. Con 8 workers la eficiencia fue de 0.5881, con 12 workers descendió a 0.4019 y con 16 workers a 0.3051. Con 32 workers, aunque el speedup permaneció alrededor de 4.87x, la eficiencia se redujo hasta 0.1521.

Por ello, el mayor número de workers no implica necesariamente la configuración más conveniente. La configuración de 16 workers obtuvo el menor tiempo absoluto, 382.95 ms, pero la configuración de 12 workers alcanzó 387.66 ms, una diferencia de aproximadamente 4.71 ms.

Esto demuestra que, a partir de cierto punto, aumentar los workers produce una ganancia temporal muy pequeña a cambio de una menor eficiencia y un mayor grado de coordinación. La determinación definitiva del punto de equilibrio requiere complementar estos resultados con el análisis del consumo de CPU, memoria y asignaciones, desarrollado en la sección 11.

10.4 Estimación de la fracción secuencial (métrica de Karp–Flatt)

Para determinar si la saturación del speedup se debe a una parte secuencial del algoritmo (ley de Amdahl) o a otros factores, se calculó la fracción secuencial determinada experimentalmente propuesta por Karp y Flatt (1990): $e = (1/S - 1/p) / (1 - 1/p)$, donde S es el speedup medido y p el número de workers.

Workers (p)	Speedup (S)	Fracción secuencial e
2	1.9655	0.0176
4	3.6615	0.0308
8	4.7050	0.1000
12	4.8224	0.1353
16	4.8817	0.1518
24	4.8603	0.1712
32	4.8668	0.1799

Tabla. Fracción secuencial experimental (Karp–Flatt) por configuración.

Si la limitación fuera únicamente una fracción secuencial fija, e se mantendría aproximadamente constante. En cambio, e crece de 0.0176 a 0.1799 conforme aumenta p . Con la fracción observada en pocos workers (entre 0.02 y 0.03), la ley de Amdahl permitiría speedups teóricos superiores a 30x, muy por encima del máximo observado de 4.88x. Por tanto, la meseta no se explica por el código secuencial, sino por un overhead que crece con p y por el límite del hardware: el procesador dispone de 6 núcleos físicos y la curva de speedup se dobla justamente al superarlos (3.66x con 4 workers frente a 4.71x con 8). Los 6 procesadores lógicos adicionales (SMT) comparten las unidades de punto flotante y el ancho de banda de memoria de su núcleo físico, un recurso crítico en el cálculo de gradientes sobre 1,682,537 registros.

11. Uso y rendimiento de recursos de cómputo

Además del tiempo de ejecución y el speedup, se analizó el comportamiento de la solución concurrente respecto al uso de recursos de cómputo. Para ello, se evaluó la utilización de CPU y se realizaron mediciones de memoria, asignaciones y operaciones de reserva dinámica para distintas cantidades de workers.

Este análisis complementa los resultados de escalabilidad presentados anteriormente y permite determinar si las pequeñas mejoras obtenidas al incrementar el número de workers justifican el costo adicional de recursos.

11.1 CPU

La ejecución concurrente fue diseñada para aprovechar los múltiples procesadores lógicos disponibles en el equipo. El sistema utilizado para las pruebas cuenta con un procesador AMD Ryzen 5 9600X, compuesto por 6 núcleos físicos y 12 procesadores lógicos.

Para observar el aprovechamiento del procesador durante una carga concurrente sostenida, se utilizó el archivo `cpu_profile.go` con una configuración de 12 workers. Esta ejecución se prolongó intencionalmente utilizando 3000 épocas, únicamente con el propósito de mantener el procesamiento activo durante el tiempo suficiente para visualizar su comportamiento mediante el Administrador de tareas de Windows.

Es importante precisar que esta ejecución extendida no fue utilizada para calcular los tiempos oficiales, speedup o eficiencia presentados en la sección 9. Los benchmarks formales se mantuvieron en 100 épocas; el perfil extendido se utilizó exclusivamente como evidencia visual del aprovechamiento de CPU.

Durante esta ejecución se observó una utilización aproximada del 92 % del procesador, con actividad distribuida entre los 12 procesadores lógicos. Este comportamiento evidencia que la implementación basada en goroutines puede traducir la concurrencia definida en el programa en procesamiento paralelo real sobre el hardware multinúcleo disponible.

Figura 11. Utilización de CPU durante la ejecución concurrente con 12 workers



Se observa una utilización aproximada del 92 % del procesador AMD Ryzen 5 9600X y actividad distribuida entre sus 12 procesadores lógicos durante la ejecución del perfil concurrente.

La evidencia muestra que los 12 workers permiten mantener una carga elevada sobre el procesador y aprovechar sus unidades lógicas de ejecución. Sin embargo, disponer de más workers que procesadores lógicos no implica disponer de más capacidad física de procesamiento, por lo que las configuraciones de 16, 24 y 32 workers dependen adicionalmente de la planificación realizada por el runtime y presentan un escenario de sobresuscripción.

11.2 Memoria

Para complementar las mediciones temporales se utilizó `resource_profile.go`, encargado de registrar el comportamiento de memoria durante el entrenamiento. Para cada configuración se realizaron tres ejecuciones y se registró el heap máximo aproximado, la memoria acumulada asignada durante la medición, el número de asignaciones dinámicas y la cantidad de ciclos del Garbage Collector.

Configuración	Peak Heap (MB)	Alloc (MB)	Mallocs	GC
Secuencial	141.92	0.04	108	0
1 worker	142.15	0.27	1,109	0
2 workers	142.36	0.49	1,609	0
4 workers	142.81	0.93	2,609	0

8 workers	143.67	1.80	4,610	0
12 workers	144.54	2.67	6,625	0
16 workers	145.35	3.47	8,633	0
24 workers	147.07	5.19	12,622	0
32 workers	148.76	6.88	16,665	0

La métrica Alloc representa la cantidad acumulada de memoria asignada durante la región medida del entrenamiento y no la memoria RAM total ocupada por la aplicación. Por ello, debe interpretarse conjuntamente con el Peak Heap y el número de asignaciones dinámicas (mallocs).

Los resultados muestran que el Peak Heap aumenta de forma moderada conforme se incrementa el número de workers, pasando de 141.92 MB en la versión secuencial a 148.76 MB con 32 workers. Esta diferencia es relativamente pequeña debido a que el conjunto de datos principal ya se encuentra cargado antes de iniciar la medición del entrenamiento.

A diferencia del Peak Heap, tanto la memoria asignada durante la medición como el número de asignaciones aumentan de manera más pronunciada al incrementar los workers. Con 12 workers se registraron 2.67 MB de asignaciones y 6,625 mallocs, mientras que con 16 workers estos valores aumentaron a 3.47 MB y 8,633 mallocs, respectivamente.

En todas las configuraciones se registró GC = 0, indicando que durante las regiones medidas del entrenamiento no se produjeron ciclos del Garbage Collector. En consecuencia, las diferencias observadas en rendimiento no pueden atribuirse a pausas de recolección de basura durante dichas mediciones.

11.3 Punto de equilibrio

La selección del punto de equilibrio no debe basarse únicamente en la configuración que obtiene el menor tiempo absoluto, sino en la relación entre rendimiento, eficiencia y utilización adicional de recursos.

Indicador	12 workers	16 workers
Media recortada	387.66 ms	382.95 ms
Speedup	4.8224x	4.8817x
Eficiencia	0.4019	0.3051

Peak Heap	144.54 MB	145.35 MB
Alloc	2.67 MB	3.47 MB
Mallocs	6,625	8,633
Relación con hardware	12 procesadores lógicos	Sobresuscripción

En términos estrictamente temporales, 16 workers constituye la configuración más rápida, con una media recortada de 382.95 ms y un speedup de 4.8817x. Sin embargo, la configuración de 12 workers obtuvo 387.66 ms y un speedup de 4.8224x, por lo que la diferencia entre ambas fue de aproximadamente 4.71 ms, equivalente a una mejora temporal de apenas 1.21 % al pasar de 12 a 16 workers.

Esta pequeña reducción del tiempo se obtiene a costa de un incremento aproximado del 30 % en la memoria acumulada asignada durante la medición y del 30 % en el número de asignaciones dinámicas, además de una reducción de la eficiencia desde 0.4019 hasta 0.3051. Asimismo, 12 workers coincide con los 12 procesadores lógicos disponibles en el equipo, mientras que 16 workers introduce sobresuscripción.

Por estas razones, se identifica 12 workers como el punto de equilibrio práctico de la solución. Esta configuración no presenta el menor tiempo absoluto, pero alcanza un rendimiento muy cercano al máximo observado, aprovecha directamente la capacidad lógica del procesador y requiere menos asignaciones que la configuración de 16 workers.

Por otro lado, 16 workers se mantiene como la configuración de mayor rendimiento temporal absoluto. La distinción entre ambos resultados permite evitar interpretar el punto de equilibrio como sinónimo de la configuración más rápida: 16 workers minimiza el tiempo, mientras que 12 workers ofrece una relación más favorable entre tiempo de ejecución, eficiencia y uso adicional de recursos.

Como complemento, se aplicaron dos criterios cuantitativos al mismo conjunto de datos. El primero toma la menor configuración que alcanza al menos el 95 % del speedup máximo: $0.95 \times 4.8817 = 4.6376$, valor que ya supera la configuración de 8 workers (4.7050x, 96.4 % del máximo). El segundo identifica la última configuración con eficiencia igual o superior a 0.5, que también corresponde a 8 workers (0.5881; con 12 workers desciende a 0.4019).

Indicador	8 workers	12 workers	Variación 8 → 12
Speedup	4.7050x	4.8224x	+2.5 %
Eficiencia	0.5881	0.4019	-31.7 %
Alloc (MB)	1.80	2.67	+48.3 %

Mallocs	4,610	6,625	+43.7 %
---------	-------	-------	---------

Tabla. Comparación entre 8 y 12 workers.

El paso de 8 a 12 workers repite el patrón observado entre 12 y 16: una mejora temporal pequeña a cambio de más asignaciones y menor eficiencia. En consecuencia, se distinguen dos resultados: 8 workers constituye el punto de equilibrio según el criterio de eficiencia, mientras que 12 workers es la configuración práctica elegida por aprovechar los 12 procesadores lógicos del equipo con un rendimiento prácticamente igual al máximo. Ambas evitan la sobresuscripción de 16, 24 y 32 workers, que no aporta mejoras significativas.

12. Trabajo en GitHub

El proyecto se aloja en un repositorio público de GitHub siguiendo el flujo Git Flow (ramas main/master y develop). Cada integrante realiza commits desde su propia cuenta para evidenciar su participación individual, conforme a lo exigido por la rúbrica.

Link del repositorio: <https://github.com/YuhiTTo/Grupo5-ProgramacionConcurrente.git>

Evidencia de commits:

The screenshot displays the GitHub Commits page for the repository `YuhiTTo/Grupo5-ProgramacionConcurrente.git`. The interface shows a list of commits on the `main` branch, filtered by 'All users' and 'All time'. The commits are grouped by date, with the most recent group being 'Commits on Sep 27, 2026'. This group includes a merge of pull request #3 from `YuhiTTo/develop` by Jaed69, and several other commits by SeuNg720p and YuhiTTo. The next group, 'Commits on Sep 13, 2026', includes a merge of pull request #2, a link to the original dataset, and several other commits by YuhiTTo and SeuNg720p. The oldest commit is from 'Commits on Sep 12, 2026', an initial commit by YuhiTTo.

El trabajo de la PC2 se integró mediante ramas feature y pull requests hacia develop, y posteriormente de develop hacia main:

- **feature/pc2-evidence (PR #4):** configuración por línea de comandos, estadística del benchmark, verificación de equivalencia, reportes y propiedades LTL.
- **feature/dataset-download (PR #5):** descarga automática y verificada del dataset.
- **develop → main (PR #6):** integración de la entrega en la rama principal.
- **feature/pc2-report-review:** resultados de verificación en SPIN y documentación de resultados en docs/pc2/.

Las cuentas de GitHub de los integrantes son Jaed69 (Jhamil Brijan Peña Cardenas), SeuNg720p (José Antonio Mayhua Hinostroza) y YuhiTTo (Lucero Salome Manchay Paredes).

13. Correcciones respecto a la PC2

Con respecto a la versión inicial del informe de la PC2, se realizaron los siguientes ajustes:

- **Correcciones de la PC1 (sección 5.2):** se agregó la lista explícita de cambios respecto a la PC1 que exige el formato del informe.
- **Verificación formal (sección 6.4):** además de las aserciones, se verificaron las propiedades LTL `safe_update`, `termination` y `mutex` para 2, 3 y 4 workers.
- **Escalabilidad (sección 10.4):** se incorporó la métrica de Karp–Flatt para explicar por qué el speedup se estanca cerca de 4.9x.
- **Punto de equilibrio (sección 11.3):** se complementó la elección de 12 workers con el criterio de eficiencia, según el cual 8 workers alcanzan el 96 % del speedup máximo.
- **Formato:** se corrigió el código de alumno de la carátula y se regeneró el índice.
- **Código:** se corrigieron los GAPs de severidad media detectados en el análisis con IA (sección 15), sin cambiar el resultado del modelo.

14. Verificación formal en SPIN

El objetivo de esta sección es demostrar formalmente que la sincronización del Worker Pool está libre de deadlocks y respeta la exclusión mutua. Se utilizó el modelo `promela/regression_workers.pml` descrito en la sección 6, verificado con SPIN 6.5.2.

14.1 Propiedades verificadas

- **Ausencia de deadlock:** corrida de seguridad (`gcc -DSAFETY -DNOCLAIM y ./pan`). SPIN explora todos los entrelazados y reporta cualquier estado final inválido, es decir, un estado en el que un proceso quedó bloqueado.
- **Exclusión mutua:** propiedad LTL `mutex, [] (in_update <= 1)`: a lo sumo un proceso actualiza los pesos a la vez, lo que justifica no usar `sync.Mutex` en Go.
- **Actualización segura:** propiedad LTL `safe_update, [] (updating -> result_count == NUM_JOBS)`: los pesos solo se actualizan después de recibir todos los resultados, como garantiza `wg.Wait()` en Go.
- **Terminación:** propiedad LTL `termination, <> done`: cada época termina.

14.2 Resultados del modelo correcto

Workers / jobs	Estados almacenados	Deadlock	mutex	safe_update	termination
2 / 4	12,358	0 errores	0 errores	0 errores	0 errores
3 / 4	145,563	0 errores	0 errores	0 errores	0 errores
4 / 4	1,080,202	0 errores	0 errores	0 errores	0 errores

Tabla. Verificación del modelo correcto en SPIN 6.5.2.

14.3 Modelos mutantes

Un resultado de cero errores solo tiene valor si SPIN sí reporta errores cuando la sincronización es incorrecta. Por eso se construyeron dos modelos mutantes, copias del modelo con un único defecto introducido a propósito, y se verificó que SPIN los rechazara.

Mutante	Defecto introducido	Chequeo	Resultado
race	Los workers también entran a la sección de actualización de pesos	LTL mutex	Violada: dos procesos en la sección crítica (in_update = 2)
race	El mismo defecto	LTL safe_update	Violada: un worker actualiza con 0 resultados recibidos
deadlock	El coordinador envía un STOP menos que la cantidad de workers	Seguridad	Deadlock detectado: un worker queda bloqueado (invalid end state)

Tabla. Mutantes detectados por SPIN (2 workers, 4 jobs).

El mismo procedimiento que rechaza los mutantes acepta el modelo correcto, por lo que el resultado de cero errores es evidencia real de que el diseño no tiene deadlocks y respeta la exclusión mutua. Como evidencia complementaria sobre el código Go, el detector de carreras (go test -race) ejecutó 80 pruebas sin detectar condiciones de carrera, incluyendo el entrenamiento concurrente con 1, 2, 4 y 8 workers. La variante de 4 workers y 8 jobs superó los 34 millones de estados sin terminar, por lo que la verificación exhaustiva se limita a configuraciones pequeñas.

15. Análisis del código con inteligencia artificial

15.1 Prompt y modelo utilizados

Se elaboró un prompt estructurado que define el rol (revisor senior de Go especializado en concurrencia), el contexto del proyecto, el alcance (todos los archivos .go, el modelo Promela y los scripts), los criterios de evaluación (calidad y mantenibilidad, seguridad, patrones de concurrencia y sincronización, rendimiento y memoria, manejo de errores, pruebas y reproducibilidad), una escala de severidad (alta, media, baja) y un formato de salida con

evidencia archivo:línea, impacto y recomendación. Se exigió que cada hallazgo estuviera respaldado por el código y que se distinguieran los hechos verificados de las hipótesis.

Se seleccionó Claude (familia Claude 5, ejecutado mediante Claude Code) porque su ventana de contexto permite leer el repositorio completo y porque puede usar herramientas para ejecutar go vet y go test durante el análisis. El prompt completo y el informe en formato Markdown se encuentran en el repositorio: docs/tp/02-prompt-analisis-ia.md y docs/tp/03-informe-gaps-ia.md.

15.2 GAPS encontrados

El análisis identificó 19 GAPS: ninguno de severidad alta, 4 de severidad media y 15 de severidad baja. Ninguno afecta la corrección del algoritmo concurrente; los de severidad media se refieren a robustez y pruebas.

GAP	Severidad	Descripción	Estado
GAP-01	Media	La descarga del dataset no tenía timeout de inactividad	Corregido
GAP-02	Media	Los errores del pipeline terminaban con código de salida 0	Corregido
GAP-03	Media	El dataset limpio se escribía de forma no atómica	Corregido
GAP-04	Media	Faltaban pruebas de preprocesamiento, escalamiento y regresión	Corregido
GAP-05	Baja	Invariante de los buffers de canales no documentada	Corregido
GAP-11	Baja	Columnas obligatorias y longitud de filas sin validar	Corregido
GAP-12	Baja	Errores de cierre de archivos ignorados	Corregido
GAP-18	Baja	Robustez de los scripts de SPIN	Corregido parcialmente
GAP-19	Baja	Formato con gofmt y ausencia de integración continua	Corregido parcialmente
Otros 10	Baja	Rendimiento, configurabilidad y duplicación de código	Mejoras futuras

Tabla. Resumen de GAPS y su estado.

15.3 Corrección de los GAPS

Las correcciones se implementaron aplicando TDD: para cada cambio de comportamiento se escribió primero una prueba que fallaba y luego la corrección. Tras los cambios, go vet no

reportó observaciones, la suite de pruebas y el detector de carreras pasaron, el modo quick mantuvo la equivalencia exacta entre las versiones secuencial y concurrente, y SPIN siguió sin errores en el modelo correcto. El análisis con IA resultó útil para revisar todo el código con un criterio uniforme, pero cada hallazgo se verificó contra el código antes de corregirlo.

16. Conclusiones y recomendaciones

16.1 Conclusiones

- **La concurrencia mejoró el tiempo sin cambiar el resultado:** con 16 workers el entrenamiento bajó de 1869.45 ms a 382.95 ms (speedup 4.88x) y ambas versiones produjeron el mismo modelo (diferencia máxima 0, $R^2 = 0.4679$).
- **El diseño evita la contención:** los workers solo leen los pesos y la actualización ocurre en un único flujo tras `wg.Wait()`, por lo que no se requirió `sync.Mutex`.
- **El límite de escalabilidad es el hardware:** la fracción secuencial de Karp–Flatt crece de 0.018 a 0.180, y el speedup se estanca al superar los 6 núcleos físicos del procesador.
- **Más workers no implica un mejor resultado:** 8 workers alcanzan el 96 % del speedup máximo con eficiencia 0.59; la sobresuscripción no aporta mejoras.
- **La verificación formal tiene valor real:** SPIN no encontró deadlocks ni violaciones de exclusión mutua en el modelo correcto y sí detectó los mutantes.

16.2 Recomendaciones

- **Mini-batch Gradient Descent** para reducir el costo por época y escalar a datasets mayores.
- **Pool de workers persistente** entre épocas para reducir asignaciones.
- **Integración continua** que ejecute `go vet`, `go test -race` y los scripts de SPIN en cada pull request.
- **Medición en otras máquinas** con más núcleos físicos para separar el efecto de SMT del overhead de coordinación.
- **Mejorar el modelo predictivo:** un R^2 de 0.47 indica que conviene explorar variables de interacción o modelos no lineales.

16.3 Limitaciones

Los tiempos se midieron en una sola máquina; la verificación en SPIN es exhaustiva solo para configuraciones pequeñas por la explosión del espacio de estados; y el análisis con IA es estático, por lo que complementa, pero no reemplaza, las pruebas y la verificación formal.

17. Anexos

- **Video de sustentación:** [ENLACE DEL VIDEO]
- **Repositorio:** <https://github.com/YuhiTTo/Grupo5-ProgramacionConcurrente>
- **Verificación formal:** docs/tp/01-verificacion-formal.md

- **Prompt e informe de GAPS:** docs/tp/02-prompt-analisis-ia.md y docs/tp/03-informe-gaps-ia.md

18. Referencias bibliográficas (formato APA)

Hellkvist, M., Özçelikkale, A., & Ahlén, A. (2021). Linear regression with distributed learning: A generalization error perspective. *IEEE Transactions on Signal Processing*, 69, 5479–5495. <https://ieeexplore.ieee.org/document/9520293>

Schüle, M. E., Lang, H., Springer, M., Kemper, A., Neumann, T., & Günnemann, S. (2022). Recursive SQL and GPU-support for in-database machine learning. *Distributed and Parallel Databases*, 40(2–3), 205–259. <https://link.springer.com/article/10.1007/s10619-022-07417-7>

Bäckström, K., Walulya, I., Papatriantafilou, M., & Tsigas, P. (2021). Consistent lock-free parallel stochastic gradient descent for fast and stable convergence. En 2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS) (pp. 423–432). IEEE. <https://ieeexplore.ieee.org/document/9460457>

Jain, R., Singh, M., Rao, A. R., & Garg, R. (2024). Predicting hospital length of stay using machine learning on a large open health dataset. *BMC Health Services Research*, 24, 860. <https://link.springer.com/article/10.1186/s12913-024-11238-y>

New York State Department of Health. (2024). Hospital inpatient discharges (SPARCS De-Identified): 2022 [Data set]. Health Data NY. https://health.data.ny.gov/Health/Hospital-Inpatient-Discharges-SPARCS-De-Identified/5dtw-tffi/about_data

Karp, A. H., & Flatt, H. P. (1990). Measuring parallel processor performance. *Communications of the ACM*, 33(5), 539–543. <https://doi.org/10.1145/78607.78614>

Holzmann, G. J. (2003). *The SPIN model checker: Primer and reference manual*. Addison-Wesley.